Majordome

Walter Dal'Maz Silva

2026-04-27

Table of contents

1 Majordome **5**
1.1 Getting Started . . . 5
1.2 Manual builds . . . 5
1.3 Developers . . . 7

I Majordome API **8**

2 Extensions **9**
2.1 Skipping execution . . . 9
2.2 Markdown and LaTeX interpolation . . . 10

3 Engineering **13**
3.1 Reactor models . . . 13
3.1.1 Warnings . . . 13
3.1.2 Utilities . . . 13
3.1.3 NormalFlowRate . . . 17
3.1.4 Plug-flow reactors . . . 19
3.1.5 PlugFlowAxialSources . . . 19
3.2 Energy models . . . 22
3.2.1 CombustionAtmosphereCHON . . . 22
3.2.2 CombustionPowerSupply . . . 22
3.2.3 Energy sources . . . 23
3.3 Numerical utilities . . . 27
3.3.1 Relaxation and convergence check utilities . . . 27
3.4 Symbolic functions . . . 32
3.4.1 PiecewiseFunction . . . 32
3.4.2 Species thermodynamics . . . 34
3.4.3 Species transport . . . 39
3.4.4 Kinetics parameterization . . . 40
3.4.5 Solution properties . . . 40
3.5 Transport properties . . . 40
3.5.1 EffectiveThermalConductivity . . . 40
3.5.2 SolutionDimless . . . 42
3.5.3 SutherlandFitting . . . 43
3.5.4 WSGGRadlibBordbar2020 . . . 45
3.6 Computer vision . . . 46

4 Simulation **47**
4.1 Elmer Multiphysics . . . 47
4.1.1 TimeStepAccumulator . . . 47
4.1.2 ElmerConvergenceData . . . 49
4.1.3 ElmerTabularData . . . 51
4.1.4 ElmerTabularMetadata . . . 52

4.2 Ansys Fluent . . . 52
4.2.1 FluentFvParticlesParser . . . 52
4.2.2 FluentInterpolationParser . . . 53
4.3 OpenFOAM . . . 54
4.4 Meshing . . . 54
4.4.1 GmshOCCModel . . . 54

5 Utilities **55**
5.1 Plotting . . . 55
5.1.1 MajordomePlot . . . 55
5.1.2 PowerFormatter . . . 61
5.1.3 General utilities . . . 62
5.2 LaTeX . . . 63
5.3 Argument parsing . . . 63
5.3.1 FuncArguments . . . 63
5.3.2 Creating class constructors . . . 65
5.4 PDF Tools . . . 66
5.4.1 PdfToTextConverter . . . 66
5.5 Misc . . . 67
5.5.1 ColorPrint . . . 67

6 Documents **68**
6.1 Tutorial . . . 69
6.2 Examples . . . 71

II Reactor Engineering **75**

7 Acetylene pyrolysis **76**
7.1 Toolbox . . . 76
7.2 Reference case . . . 76
7.3 Scan table . . . 78

8 Counter-current PFR **80**
8.1 Introduction . . . 80
8.2 Shared functionalities . . . 81
8.3 Single plug-flow reactor . . . 81
8.3.1 Example - single PFR simulation . . . 81
8.4 Counter-current flow reactors . . . 83
8.4.1 Example - pair of coupled PFR . . . 83
8.5 Adding losses to the environment . . . 84
8.5.1 Example - environment coupling . . . 85
8.6 Registering flow for a single reactor . . . 86
8.6.1 Example - prototype with a single reactor . . . 86
8.7 Registering flow for a pair of reactors . . . 87
8.7.1 Example - extension to coupled system . . . 87

9 Fitting NASA9 data **91**
9.1 Helper utilities . . . 91
9.2 Nitrogen actual fit . . . 91
9.3 Enforcing constant specific heat . . . 92
9.4 Ensuring enthalpy continuity . . . 94

9.5 Application to a real material 95
9.6 Generalizing approach 98

References **104**

1 Majordome

1.1 Getting Started

If you do not intend to build the package, you can find us on Majordome on PyPI and install it with pip:

```
pip install majordome
```

To register `majordome` as a Jupyter kernel, run:

```
majordome --install-kernel
```

You can customize the kernel identity if needed:

```
majordome --install-kernel `
    --kernel-name majordome-dev `
    --display-name "Majordome (dev)"
```

1.2 Manual builds

If you don't fit in the first group, you can build the package from source.

The environment for developing Majordome requires at least a reasonably mondern version of Python and Rust; documentation depends on Quarto and `uv` is used for managing the environment. The exact versions may change over time, but as of the last update, the following (minimum) versions are recommended:

- Python 3.12+
- Rust toolchain
- Quarto CLI
- uv

> **Tip**
>
> Ubuntu users can bootstrap the environment with the following commands; make sure to check the latest versions of the tools and update the commands accordingly.
>
> ```
> sudo apt-get install -y python3-dev
>
> curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh -s -- -y
> source "$HOME/.cargo/env"
>
> curl -LsSf https://astral.sh/uv/install.sh | sh
> ```

```
source "$HOME/.local/bin/env"

quarto_repo="https://github.com/quarto-dev/quarto-cli"
quarto_deb="quarto-1.9.37-linux-amd64.deb"

wget $quarto_repo/releases/download/v1.9.37/$quarto_deb
sudo apt-get install -y ./$quarto_deb
```

The following steps illustrate how to build the package, generate the documentation, and publish it to GitHub Pages. The commands are provided for both Windows PowerShell and Unix-like shells; make sure to use the appropriate ones for your system.

- Build the package:

```
uv build --wheel
```

- Inspect the wheel (especially if new data was included):

```
python -m zipfile -l dist/*.whl
```

- Create and a ctivate a virtual environment:

```
uv venv --python python3.12 .venv

# Activate on Windows:
. .venv/Scripts/activate

# Activate on Unix:
. .venv/bin/activate
```

- Install the package (remove `[docs]` to expand the argument, then add it back to install the dependencies before generating the documentation):

```
uv pip install ./dist/*.whl[docs]
```

- Point Quarto to the virtual environment:

```
# Windows:
$env:QUARTO_PYTHON = ".venv/Scripts/python.exe"

# Unix:
export QUARTO_PYTHON=".venv/bin/python"
```

- Generate the docs:

```
quarto render docs
```

1.3 Developers

The development focuses on portability and ease of setup, so by default any dependency relying on a C/C++ compiler is forbidden. This means that the development environment can be set up on any platform without needing to install additional compilers or tools.

Majordome code is documented using Numpydoc style docstrings, and the documentation is generated by module `majordome.documents` using Quarto markdown. This design choice reflects the academic aim of the package, with reporting and notebook support as first-class citizens.

In addition to what is described in the manual builds section, developers can also need to publish the docs, update the version, and publish the new version to GitHub and PyPI. This is fully automated by `release.py` to ensure consistency in actions, simple run one of the following:

```
python release.py --version 1.0.0
python release.py --version patch
python release.py --version minor
python release.py --version major
```

Nonetheless, the following commands are documented to keep track of what is being automated. Please notice that everything related to virtual environment handling is ommitted for simplicity, as super-users should be able to handle it on their own:

```
# Publish the docs to GitHub Pages (owners only):
quarto publish gh-pages --no-prompt --no-browser docs

# Install (the first time) cargo-edit for Cargo.toml:
cargo install cargo-edit

# Manually set the new version:
# (remove `--dry-run` to actually update the version)
uv version 1.0.0 --dry-run
cargo set-version 1.0.0 --dry-run

# Or use the bump to automatically increment:
uv version --bump patch --dry-run
uv version --bump minor --dry-run
uv version --bump major --dry-run
cargo set-version --bump patch --dry-run
cargo set-version --bump minor --dry-run
cargo set-version --bump major --dry-run

# Push the new version to GitHub (DO NOT COPY THIS):
gh release create v1.0.0 dist/*.whl --generate-notes
```

- Publish the new version to PyPI (owners only) is done manually with `uv`:

```
uv publish
```

Part I

Majordome API

2 Extensions

aka majordome.magic

On top of your notebook, simply load the extension:

```
%load_ext majordome.magic
```

For JupyterLab 4, a dedicated frontend extension is now available in this repository at `jupyterlab/majordome-mdmagic`.

```
cd jupyterlab/majordome-mdmagic
npm install
npm run build
jupyter labextension link .
```

After this, start JupyterLab as usual and the extension will switch `%%md` code cells to Markdown highlighting (or LaTeX for pure `$$...$$` display-math bodies).

2.1 Skipping execution

> **SkipperMagic**
>
> `SkipperMagic():`
>
> An extension for skipping the execution of code cells.

The module provides a simple extension for skipping the execution of code cells in a notebook. It does that and only that. The goal is to control notebook execution from the environment, which is useful when running notebooks with long-running cells, as numerical simulations.

```
%skipper_verbosity 1
```

```
Setting skipper verbosity to 1
```

Here we will use `sleep` to emulate a long-running cell, and we will control its execution with an environment variable. Generally speaking, the goal is to export environment variables in the terminal before running the notebook, but for demonstration purposes, we will set it in the notebook itself.

```
from time import sleep
import os
```

The following (case insensitive) options are accepted by the `%%skipper` magic command. Please, notice that the *truthy* values mean *skip this cell* (as the extension name implies, but even myself can be confused by the double negative here).

```
{'true': True, '1': True, 'yes': True, 'false': False, '0': False, 'no': False}
```

Below we set the environment variable `SOLVE_LONNNNG` to `true`, which means that the cell will be skipped. You can change it to `false` to execute the cell, or even set it to any other value, as long as it is in the lookup table.

```
os.environ["SOLVE_LONNNNG"] = "true"
```

```
did_you_skip_it = True
```

If everything is file, the following cell will be skipped:

```
%%skipper SOLVE_LONNNNG
sleep(100)
did_you_skip_it = False
```

```
Skipping cell: SOLVE_LONNNNG=True
```

And here we check if it was actually skipped:

```
print("Did you skip it?", did_you_skip_it)
```

```
Did you skip it? True
```

You can also avoid the *Skipping cell* message by setting the verbosity to 0:

```
%skipper_verbosity 0
```

```
%%skipper SOLVE_LONNNNG
sleep(100)
did_you_skip_it = False
```

2.2 Markdown and LaTeX interpolation

MdMagic

`MdMagic()`:

An extension for interpolating code in Markdown.
Cell magic extension to interpolate Markdown and LaTeX. It must be used from code cells and the content is rendered as Markdown. As such, it is expected to be used only when required, as without some frontend tweaks you will have undesirable syntax highlighting in the cell, as it is not interpreted as text. Nonetheless, this proves very useful for complex reporting with dynamic values.

- Variables are interpolated from the IPython user namespace using Python format placeholders, e.g. `{variable}`.
- If a variable is not found or evaluation fails, the original placeholder is left unchanged in the output, e.g. `{variable}`. For details, check `eval_placeholder` method.

- Curly braces inside LaTeX math blocks (`$...$` or `$$...$$`) are automatically escaped before interpolation so expressions such as `$x_{i}$` render correctly.
- You can override the interpretation of curly braces inside LaTex math blocks by providing a format spec, e.g. `{variable:.2f}`.

Declare some values in the user namespace:

```
variable = "hello"
some_value = 12.2
```

Below we have a Markdown cell with some variables and LaTeX expressions. The variables will be replaced by their values, and the LaTeX will be rendered correctly.

```
%%md
I have this string with a variable: {variable}.

There is also some latex: $\int_a^b f(x) \; dx$.

- Some formatter value {some_value:.2f}

- Some block math:

$$
\int_0^\infty e^{-x^2} dx = \frac{\sqrt{\pi}}{2}
$$
```

I have this string with a variable: hello.

There is also some latex: $\int_a^b f(x) \; dx$.

- Some formatter value 12.20
- Some block math:

$$
\int_0^\infty e^{-x^2} dx = \frac{\sqrt{\pi}}{2}
$$

! About Quarto rendering

Please notice that when working in VSCode Jupyter extension, the following currently fails, *i.e.* it does not recognize the `%%md` magic command as it is not on the top of the cell; if you change the order, the Quarto annotation will be interpreted as markdown. For properly rendering Quarto documents, consider adding a global configuration for the echo option set to `false` and explicitly set it to `true` in the cells where you want to see the code. This is a known issue that will (maybe) be fixed in the future, but for now, it is a good workaround (that actually enforces a good practice of hiding code cells in the final document, which is what you usually want when writing reports).

```
#| echo: false
%%md
```

The following is also possible in math blocks:

```
%%md

$$
F(x) = {some_value:.2f} \cdot x^2
$$

But there is no way to use the plain `some_value` without a format specifier, as it would be

$$
F(x) = {some_value} \cdot x^2
$$
```

$$F(x) = 12.20 \cdot x^2$$

But there is no way to use the plain `some_value` without a format specifier, as it would be indistinguishable from a regular LaTeX expression.

$$F(x) = some_value \cdot x^2$$

SymPy expressions are also supported, and they will be rendered as LaTeX. In general rendering is done by displaying some expression as provided below:

```
import sympy as sp

x = sp.symbols("x")
f_x = some_value * x**2
f_x
```

$12.2x^2$

With help of the extension we can also render it in markdown:

Now we can also talk about the function we just defined: $12.2x^2$! If you need to inject the expression inside a math block, you can do so by calling `sp.latex` and using the string formatter: $f = 12.2x^2$. It should also work with block math:

$$f = 12.2x^2$$

3 Engineering

aka majordome.engineering

3.1 Reactor models

3.1.1 Warnings

Module warnings can be controlled throught the following attributes:

toggle_reactor_warnings

```
toggle_reactor_warnings(**kwargs : Any): -> None:
```

Reverse truth value of warning flags.

Parameters

```
toggle_non_key_value : bool | None = None
```

If true, reverse truth value of warning about non key-value composition.

```
toggle_missing_species_name : bool | None = None
```

If true, reverse truth value of warning about missing species name.

```
toggle_unknown_species : bool | None = None
```

If true, reverse truth value of warning about unknown species.

3.1.2 Utilities

Let's start by creating a standard Cantera solution:

```
solution = ct.Solution("airish.yaml")
solution.TPY = 273.15, ct.one_atm, "N2: 0.78 O2: 0.21, AR: 0.01"
```

Conversion of composition strings with name filtering is available:

composition_to_dict

```
composition_to_dict(
    Y : str,
    species_names : list[str] = []
    ) -> dict[str, float]:
```

Convert a Cantera composition string to dictionary.

Parameters

`Y : str`

Composition specification string, *e.g.* "O2: 1, N2: 3".

`species_names : list[str] = []`

List of valid species names for validation. If provided, only species in this list will be included in the output dictionary. If not provided, all species will be included.

```
composition_to_dict("O2: 1, teste: 1", solution.species_names)
```

```
{'O2': 1.0}
```

It is also possible to set unit composition with species names; if you do not provide a validation list, all are are kept:

```
composition_to_dict("O2, N2, hello")
```

```
{'O2': 1.0, 'N2': 1.0, 'hello': 1.0}
```

When working with arrays, take care not to end up in the following situation:

```
composition_to_array(", teste: 1", solution.species_names)
```

```
array([0., 0., 0.])
```

There is also a helper for generating inputs for use with `tabulate.tabulate`:

solution_report

```
solution_report(
    sol : cantera.composite.Solution | cantera.composite.Quantity,
    specific_props : bool = True,
    composition_spec : str = 'mass',
    selected_species : list[str] = [],
    **kwargs : Any
```

```
    ) -> list[tuple[str, str, typing.Any]]:
```

Generate a solution report for tabulation.

Parameters

`sol : cantera.composite.Solution | cantera.composite.Quantity`

Cantera solution object for report generation.

`specific_props : bool = True`

If true, add specific heat capacity and enthalpy.

`composition_spec : str = 'mass'`

Composition units specification, `mass` or `mole`.

`selected_species : list[str] = []`

Selected species to display; return all if a composition specification was provided.

`show_mass : bool  = 'False'`

If true, add mass of quantity to report if `sol` is a `ct.composite.Quantity`.

```
data = solution_report(solution, specific_props=True,
                       composition_spec="mass", selected_species=[])
print(tabulate(data))
```

```
----------------------  --------  ------------
Temperature             K            273.15
Pressure                Pa        101325
Density                 kg/m³          1.28735
Specific enthalpy       J/(kg.K)  -25114.9
Specific heat capacity  J/(kg.K)    1004.95
mass: AR                -              0.01
mass: N2                -              0.78
mass: O2                -              0.21
----------------------  --------  ------------
```

Because sometimes Cantera lacks hard-copy utilities for certain classes, we provide simple wrappers that create new instances and set the state to the same of the source object. Nothing checked, nothing tested. A first of this kind is `copy_solution`, illustrated below:

copy_solution

`copy_solution(sol : Solution): -> Solution:`

Makes a hard copy of a Solution object.

Parameters

`sol : Solution`

Solution to be copied.

```
newairs = copy_solution(solution)
newairs.TPX = 373.15, None, newairs.X
print(tabulate(solution_report(newairs)))
```

```
----------------------  --------  -------------
Temperature             K            373.15
Pressure                Pa        101325
Density                 kg/m³          0.942356
Specific enthalpy       J/(kg.K)   75902.1
Specific heat capacity  J/(kg.K)    1015.82
mass: AR                -              0.01
mass: N2                -              0.78
mass: O2                -              0.21
----------------------  --------  -------------
```

```
print(tabulate(solution_report(solution)))
```

```
----------------------  --------  ------------
Temperature             K            273.15
Pressure                Pa        101325
Density                 kg/m³          1.28735
Specific enthalpy       J/(kg.K)  -25114.9
Specific heat capacity  J/(kg.K)    1004.95
mass: AR                -              0.01
mass: N2                -              0.78
mass: O2                -              0.21
----------------------  --------  ------------
```

In addition to this, there is `copy_quantity`, which proves quite useful in establishing an *algebra* of mixtures.

copy_quantity

`copy_quantity(qty : Quantity): -> Quantity:`

Makes a hard copy of a ct.composite.Quantity object.

Parameters

`qty : Quantity`

Quantity to be copied.

```
air = copy_solution(solution)

air1 = ct.Quantity(air, mass=1.0)
air1.TPX = 373.15, None, newairs.X

air2 = copy_quantity(air1)
air2.TPX = 273.15, None, air2.X

mixair = air1 + air2
air1.T, air2.T, mixair.T
```

```
(373.15, 273.15, 323.337485365699)
```

3.1.3 NormalFlowRate

Common daily work activity for the process engineer is to perform mass balances, but wait, ..., gas flow rates are generally provided under normal conditions, and compositions may vary, so you need to compute normal densities first... whatever. This class provides a calculator wrapping a Cantera solution object so that your life gets easier.

NormalFlowRate

```
NormalFlowRate(
    mech : str | pathlib.Path,
    *,
    X : str | dict[str, float] | None = None,
    Y : str | dict[str, float] | None = None,
    T_ref : float = 273.15,
    P_ref : float = 101325.0,
    name : str | None = None
    ) -> None:
```

Compute normal flow rate for a given composition.
This class makes use of the user defined state to create a function object that converts industrial scale flow rates in normal cubic meters per hour to kilograms per second. Nothing more, nothing less, it aims at helping the process engineer in daily life for this quite repetitive need when performing mass balances.

Parameters

`mech : str | pathlib.Path`

Path to Cantera mechanism used to compute mixture properties.

```
X : str | dict[str, float] | None = None
```

Composition specification in mole fractions. Notice that both `X` and `Y` are mutally exclusive keyword arguments.

```
Y : str | dict[str, float] | None = None
```

Composition specification in mass fractions. Notice that both `X` and `Y` are mutally exclusive keyword arguments.

```
T_ref : float = 273.15
```

Reference temperature of the system. If your industry does not use the same standard as the default values, and only in that case, please consider updating this keyword.

```
P_ref : float = 101325.0
```

Reference pressure of the system. If your industry does not use the same standard as the default values, and only in that case, please consider updating this keyword.

```
name : str | None = None
```

Name of phase in mechanism, if more than one are specified within the same Cantera YAML database file.

Its simples use case is as follows:

```
nfr = NormalFlowRate("airish.yaml")
print(f"Convert 1000 Nm³/h to {nfr(1000.0):.5f} kg/s")
```

```
Convert 1000 Nm³/h to 0.35903 kg/s
```

If the database file default composition does not suit you, no problems:

```
nfr = NormalFlowRate("airish.yaml", X="N2: 1")
print(f"Convert 1000 Nm³/h to {nfr(1000.0):.5f} kg/s")
```

```
Convert 1000 Nm³/h to 0.34718 kg/s
```

You can also print a nice report to inspect the details of internal state. For more, please check its API documentation.

```
print(nfr.report())
```

```
|------------------------|----------|--------------|
| Temperature            | K        |    273.15    |
| Pressure               | Pa       | 101325       |
| Density                | kg/m³    |      1.24985 |
| Specific enthalpy      | J/(kg.K) | -25864.9     |
| Specific heat capacity | J/(kg.K) |   1035.52    |
| mass: N2               | -        |      1       |
```

3.1.4 Plug-flow reactors

3.1.5 PlugFlowAxialSources

PlugFlowAxialSources

```
PlugFlowAxialSources(
    n_reactors : int,
    n_species : int
    ) -> None:
```

Provides a data structure for use with `PlugFlowChainCantera`.
Helper data class for use with the solution method `loop` of the plug-flow reactor implementation.
It provides the required memory for storage of source terms distributed along the reactor.

Parameters

`n_reactors : int`

Number of reactors in chain.

`n_species : int`

Number of species in mechanism.

`Q : NDArray[np.float64] = None`

Array of external heat source [W].

`m : NDArray[np.float64] = None`

Array of axial mass source terms [kg/s].

`h : NDArray[np.float64] = None`

Array of enthalpy of axial mass source terms [J/kg].

`Y : NDArray[np.float64, np.float64] = None`

Array of mass fractions of axial mass source terms [-].

3.1.5.1 PlugFlowChainCantera

PlugFlowChainCantera

```
PlugFlowChainCantera(
    mechanism : str,
    phase : str,
    z : numpy.ndarray[tuple[typing.Any, ...], numpy.dtype[numpy.float64]],
    V : numpy.ndarray[tuple[typing.Any, ...], numpy.dtype[numpy.float64]],
    P : float = 101325.0,
```

```
    K : float = 1.0,
    smoot_flux : bool = False,
    cantera_steady : bool = True
    ) -> None:
```

Plug-flow reactor as a chain of 0-D reactors with Cantera.

Parameters

`mechanism : str`

Name or path to Cantera mechanism to be used.

`phase : str`

Name of phase to simulate (not inferred, even if a single is present!).

`z : numpy.ndarray[tuple[typing.Any, ...], numpy.dtype[numpy.float64]]`

Spatial coordinates of reactor cells [m].

`V : numpy.ndarray[tuple[typing.Any, ...], numpy.dtype[numpy.float64]]`

Volumes of reactor cells [m^3].

`P : float = 101325.0`

Reactor operating pressure [Pa].

`K : float = 1.0`

Valve response constant (do not use unless simulation fails).

`smoot_flux : bool = False`

Apply a smoot transition function when internal stepping is performed; this is intended to avoid unphysical steady state approximations.

`cantera_steady : bool = True`

If true, use Cantera' s `advance_to_steady_state` to solve problem; otherwise advance over meaninful time-scale of the problem.

get_reactor_data

`get_reactor_data(pfr : PlugFlowChainCantera): -> PlugFlowAxialSources:`

Wrapper to allocate properly dimensioned solver data.

Parameters

`pfr : PlugFlowChainCantera`

Reactor for which data is to be allocated.

3.2 Energy models

This module is concerned with providing classes for computation of energy sources; these are mostly related to estimation of fuel consumption or integration to another simulation modules. Hereafter we illustrate the in cascading complexity the available energy sources. Some of these are simple wrapper around features provided by the combustion utilities.

3.2.1 CombustionAtmosphereCHON

A common need in industry is to evaluate the real heating value from a gas composition reported by the supplier. This is the main goal of `CombustionAtmosphereCHON`, which has a very simple API provided below:

```
gas = CombustionAtmosphereCHON("gri30.yaml")
lhv = gas.solution_heating_value("CH4: 1", "O2: 1")
print(f"Solution heating value: {lhv:.2f} MJ/kg")
```

```
Solution heating value: 50.03 MJ/kg
```

3.2.2 CombustionPowerSupply

Built upon `CombustionAtmosphereCHON`, `CombustionPowerSupply` performs the basic calculations for retrieving required flow rates from a given power specification; this proves very handy for the CFD engineer.

```
supply = CombustionPowerSupply(500.0, 1.0, "CH4: 1", "O2: 1", "gri30.yaml")
print(supply.report())
```

```
| Property                  | Unit    |     Value |
|---------------------------|---------|-----------|
| Required power            | kW      | 500       |
| Lower heating value       | MJ/kg   |   50.0254 |
| Fuel mass flow rate       | kg/h    |   35.9817 |
| Oxidizer mass flow rate   | kg/h    | 143.532   |
```

```
| Total mass flow rate      | kg/h    | 179.514   |
| Fuel volume flow rate     | Nm³/h   |  50.2707  |
| Oxidizer volume flow rate | Nm³/h   | 100.541   |
| Total volume flow rate    | Nm³/h   | 150.812   |
| Water production          | kg/h    |  80.8092  |
| Carbon dioxide production | kg/h    |  98.7047  |
| Total emissions           | kg/h    | 179.514   |
```

3.2.3 Energy sources

3.2.3.1 HeatedGasEnergySource

```python
source = HeatedGasEnergySource("airish.yaml", 500.0, mass_flow_rate=1.0)
print(source.report())
```

```
| Property                | Unit      | Value                  |
|-------------------------|----------|-----------------------|
| Source kind             |          | HeatedGasEnergySource |
| Provided power          | kW       | 500.0                 |
| Source                  |          | airish.yaml           |
| Phase                   |          | air                   |
| Reference area          | m²       | 1.0                   |
| Mass flow rate          | kg/s     | 1.0                   |
| Volume flow rate        | m³/s     | 2.2051590808893957    |
| Momentum flux           | kg.m/s²  | 2.2051590808893957    |
| Reference temperature   | K        | 298.15                |
| Reference pressure      | Pa       | 101325.0              |
| Temperature             | K        | 778.5219246537463     |
| Pressure                | Pa       | 101324.99999999999    |
| Density                 | kg/m³    | 0.453482022529039     |
| Specific enthalpy       | J/(kg.K) | 500038.49937969144    |
| Specific heat capacity  | J/(kg.K) | 1091.748083028256     |
| mass: AR                | -        | 0.013790127718329308  |
| mass: N2                | -        | 0.7542602692440457    |
| mass: O2                | -        | 0.23194960303762516   |
```

```python
source = HeatedGasEnergySource("airish.yaml", 500.0, mass_flow_rate=1.0,
                               cross_area=0.1, Y="N2: 0.79, O2: 0.21")
print(source.report())
```

```
| Property                | Unit      | Value                  |
|-------------------------|----------|-----------------------|
| Source kind             |          | HeatedGasEnergySource |
| Provided power          | kW       | 500.0                 |
| Source                  |          | airish.yaml           |
| Phase                   |          | air                   |
| Reference area          | m²       | 0.1                   |
| Mass flow rate          | kg/s     | 1.0                   |
| Volume flow rate        | m³/s     | 2.2090693490835482    |
| Momentum flux           | kg.m/s²  | 22.090693490835484    |
```

```
| Reference temperature  | K         | 298.15                |
| Reference pressure     | Pa        | 101325.0              |
| Temperature            | K         | 774.4142430033143     |
| Pressure               | Pa        | 101324.99999999999    |
| Density                | kg/m³     | 0.45267931512193527   |
| Specific enthalpy      | J/(kg.K)  | 500040.3236104384     |
| Specific heat capacity | J/(kg.K)  | 1100.3346580471573    |
| mass: N2               | -         | 0.79                  |
| mass: O2               | -         | 0.21                  |
```

3.2.3.2 CombustionEnergySource

```
fuel_state = StateType("CH4: 1",             300, 101325)
oxid_state = StateType("N2: 0.79, O2: 0.21", 300, 101325)

ops = CombustionPowerOp(500.0, 1.0, fuel_state, oxid_state)
source = CombustionEnergySource("ch4/bfer.yaml", operation=ops, cross_area=0.1)

# Recover values for other tests that follow
# TODO wrap these in properties for clean API:
mdot_fuel = source._qty_fuel.mass
mdot_oxid = source._qty_oxid.mass

nfr_fuel = NormalFlowRate.new_from_solution(source._qty_fuel)
nfr_oxid = NormalFlowRate.new_from_solution(source._qty_oxid)

qdot_fuel = 3600 * mdot_fuel / nfr_fuel.density
qdot_oxid = 3600 * mdot_oxid / nfr_oxid.density

print(source.report())
```

```
| Property               | Unit      | Value                  |
|------------------------|----------|------------------------|
| Source kind            |          | CombustionEnergySource |
| Provided power         | kW       | 500.0                  |
| Source                 |          | ch4/bfer.yaml          |
| Phase                  |          | CH4_BFER               |
| Reference area         | m²       | 0.1                    |
| Mass flow rate         | kg/s     | 0.18117768922524702    |
| Volume flow rate       | m³/s     | 1.2210278725486656     |
| Momentum flux          | kg.m/s²  | 2.2122300842798666     |
| ******* FLUE:          |          |                        |
| Temperature            | K        | 2257.7969133307734     |
| Pressure               | Pa       | 101325.00000819027     |
| Density                | kg/m³    | 0.14838128866548536    |
| Specific enthalpy      | J/(kg.K) | -254492.89776989678    |
| Specific heat capacity | J/(kg.K) | 1516.597922218735      |
| mass: CH4              | -        | 2.7662274427222582e-17 |
| mass: CO               | -        | 0.010559409120413024   |
| mass: CO2              | -        | 0.13474113875913193    |
| mass: H2O              | -        | 0.12389490081229192    |
```

```
| mass: N2                | -         | 0.7247731344386522     |
| mass: O2                | -         | 0.006031416869510841   |
| ******* FUEL:           |           |                        |
| Temperature             | K         | 300.0                  |
| Pressure                | Pa        | 101325.00000000001     |
| Density                 | kg/m³     | 0.6516985521312658     |
| Specific enthalpy       | J/(kg.K)  | -4645856.881890704     |
| Specific heat capacity  | J/(kg.K)  | 2229.0429122783953     |
| mass: CH4               | -         | 1.0                    |
| ******* OXIDIZER:       |           |                        |
| Temperature             | K         | 300.0                  |
| Pressure                | Pa        | 101325.00000000001     |
| Density                 | kg/m³     | 1.171970349439655      |
| Specific enthalpy       | J/(kg.K)  | 1907.6015935135354     |
| Specific heat capacity  | J/(kg.K)  | 1010.0686132988887     |
| mass: N2                | -         | 0.7670907820415769     |
| mass: O2                | -         | 0.2329092179584231     |
```

```
ops = CombustionFlowOp("mass", mdot_fuel, mdot_oxid, fuel_state, oxid_state)
source = CombustionEnergySource("ch4/bfer.yaml", operation=ops, cross_area=0.1)
print(source.report())
```

```
| Property                | Unit      | Value                  |
|-------------------------|-----------|------------------------|
| Source kind             |           | CombustionEnergySource |
| Provided power          | kW        | 500.0                  |
| Source                  |           | ch4/bfer.yaml          |
| Phase                   |           | CH4_BFER               |
| Reference area          | m²        | 0.1                    |
| Mass flow rate          | kg/s      | 0.18117768922524702    |
| Volume flow rate        | m³/s      | 1.2210278725486656     |
| Momentum flux           | kg.m/s²   | 2.2122300842798666     |
| ******* FLUE:           |           |                        |
| Temperature             | K         | 2257.7969133307734     |
| Pressure                | Pa        | 101325.00000819027     |
| Density                 | kg/m³     | 0.14838128866548536    |
| Specific enthalpy       | J/(kg.K)  | -254492.89776989678    |
| Specific heat capacity  | J/(kg.K)  | 1516.597922218735      |
| mass: CH4               | -         | 2.7662274427222582e-17 |
| mass: CO                | -         | 0.010559409120413024   |
| mass: CO2               | -         | 0.13474113875913193    |
| mass: H2O               | -         | 0.12389490081229192    |
| mass: N2                | -         | 0.7247731344386522     |
| mass: O2                | -         | 0.006031416869510841   |
| ******* FUEL:           |           |                        |
| Temperature             | K         | 300.0                  |
| Pressure                | Pa        | 101325.00000000001     |
| Density                 | kg/m³     | 0.6516985521312658     |
| Specific enthalpy       | J/(kg.K)  | -4645856.881890704     |
| Specific heat capacity  | J/(kg.K)  | 2229.0429122783953     |
| mass: CH4               | -         | 1.0                    |
| ******* OXIDIZER:       |           |                        |
```

```
| Temperature            | K        | 300.0                  |
| Pressure               | Pa       | 101325.00000000001     |
| Density                | kg/m³    | 1.171970349439655      |
| Specific enthalpy      | J/(kg.K) | 1907.6015935135354     |
| Specific heat capacity | J/(kg.K) | 1010.0686132988887     |
| mass: N2               | -        | 0.7670907820415769     |
| mass: O2               | -        | 0.2329092179584231     |
```

```
ops = CombustionFlowOp("volume", qdot_fuel, qdot_oxid, fuel_state, oxid_state)
source = CombustionEnergySource("ch4/bfer.yaml", operation=ops, cross_area=0.1)
print(source.report())
```

```
| Property               | Unit     | Value                  |
|------------------------|----------|------------------------|
| Source kind            |          | CombustionEnergySource |
| Provided power         | kW       | 500.0                  |
| Source                 |          | ch4/bfer.yaml          |
| Phase                  |          | CH4_BFER               |
| Reference area         | m²       | 0.1                    |
| Mass flow rate         | kg/s     | 0.18117768922524705    |
| Volume flow rate       | m³/s     | 1.2210278725486658     |
| Momentum flux          | kg.m/s²  | 2.212230084279867      |
| ******* FLUE:          |          |                        |
| Temperature            | K        | 2257.7969133307734     |
| Pressure               | Pa       | 101325.00000819027     |
| Density                | kg/m³    | 0.14838128866548536    |
| Specific enthalpy      | J/(kg.K) | -254492.89776989678    |
| Specific heat capacity | J/(kg.K) | 1516.597922218735      |
| mass: CH4              | -        | 2.7662274427222582e-17 |
| mass: CO               | -        | 0.010559409120413024   |
| mass: CO2              | -        | 0.13474113875913193    |
| mass: H2O              | -        | 0.12389490081229192    |
| mass: N2               | -        | 0.7247731344386522     |
| mass: O2               | -        | 0.006031416869510841   |
| ******* FUEL:          |          |                        |
| Temperature            | K        | 300.0                  |
| Pressure               | Pa       | 101325.00000000001     |
| Density                | kg/m³    | 0.6516985521312658     |
| Specific enthalpy      | J/(kg.K) | -4645856.881890704     |
| Specific heat capacity | J/(kg.K) | 2229.0429122783953     |
| mass: CH4              | -        | 1.0                    |
| ******* OXIDIZER:      |          |                        |
| Temperature            | K        | 300.0                  |
| Pressure               | Pa       | 101325.00000000001     |
| Density                | kg/m³    | 1.171970349439655      |
| Specific enthalpy      | J/(kg.K) | 1907.6015935135354     |
| Specific heat capacity | J/(kg.K) | 1010.0686132988887     |
| mass: N2               | -        | 0.7670907820415769     |
| mass: O2               | -        | 0.2329092179584231     |
```

3.3 Numerical utilities

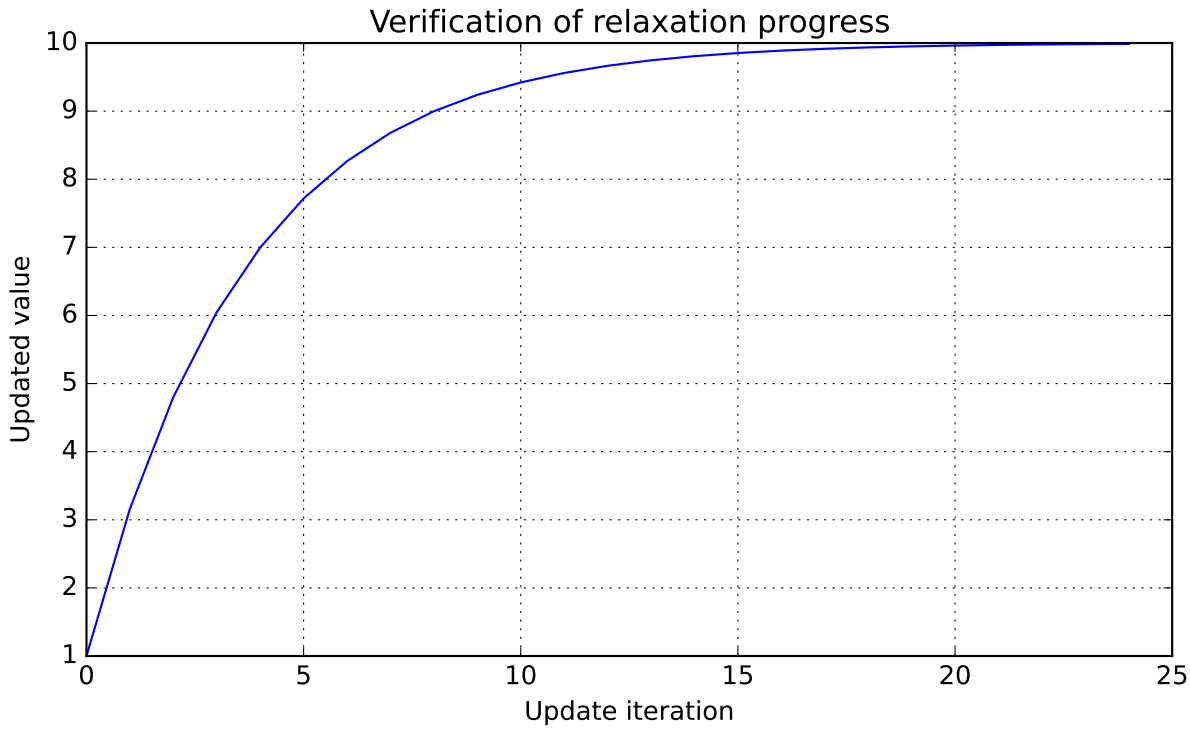
3.3.1 Relaxation and convergence check utilities

Simplest of relaxation methods; assume you have a new updated solution $A^{\star}_{new}$ for a problem whose past state was A_{old}, then the manager will ensure the following relaxation will be applied to compute the next solution state A_{new} to be used in whatever you are computing:

$$A_{new} = \alpha A_{old} + (1-\alpha) A^{\star}_{new}$$
$$A_{old} = A_{new}$$

Notice that in this formulation, α (or `alpha` in the API) represents the fraction of old solution to be used in the *smearing* process. Below we illustrate the effect of a step function H valued at 10 from the begining over consecutive updates (here we do not test for convergence, as that is problem specific and for this simple case the required number of steps could be evaluated by hand, take some time to try!).

```
alpha = 0.76
niter = 50

single = np.ones(1)
relaxer = RelaxUpdate(single, alpha)

opts = dict(n_vars=1, max_iter=niter, patience=3, rtol=0.001)
converged = StabilizeNvarsConvergenceCheck(**opts)

history = np.zeros(niter+1)
history[0] = single[0]

H = np.asarray([10])

for n in range(niter):
    single[:] = relaxer(H)
    history[n+1] = single[0]

    if converged(single[0]):
        history = history[:n+2]
        break

_ = plot_history(history)
```



3.3.1.1 RelaxUpdate

RelaxUpdate

```
RelaxUpdate(
    v_ini : numpy.ndarray[tuple[typing.Any, ...], numpy.dtype[numpy.float64]],
    alpha : float = 0.5
    ) -> None:
```

Relax solution for updating new iteration.

Parameters

`v_ini : numpy.ndarray[tuple[typing.Any, ...], numpy.dtype[numpy.float64]]`

Initial guess of solution.

`alpha : float = 0.5`

Fraction of old solution to use at updates.

update

```
update(
    self : Any,
    alpha : float
    ) -> None:
```

Update relaxation coefficients.

Parameters

```
alpha : float
```

Fraction of old solution to use at updates.

call

```
__call__(
    self : Any,
    v_new : numpy.ndarray[tuple[typing.Any, ...], numpy.dtype[numpy.float64]]
    ) -> numpy.ndarray[tuple[typing.Any, ...], numpy.dtype[numpy.float64]]:
```

Evaluate new relaxed solution estimate.

Parameters

```
v_new : numpy.ndarray[tuple[typing.Any, ...], numpy.dtype[numpy.float64]]
```

New solution estimate to be relaxed.

3.3.1.2 StabilizeNvarsConvergenceCheck

StabilizeNvarsConvergenceCheck

```
StabilizeNvarsConvergenceCheck(
    *,
    n_vars : int,
    min_iter : int = 1,
    max_iter : int = 1000000,
    patience : int = 10,
    rtol : float = 1e-10,
    atol : float = 1e-20,
    equal_nan : bool = False,
    log_iter : bool = False
```

```
    ) -> None:
```

Check stabilization towards a constant value along iterations.

Parameters

`n_vars : int`

Number of variables to be checked in problem.

`min_iter : int = 1`

Minimum number of iterations before considering converged.

`max_iter : int = 1000000`

Maximum number of iterations before considering failure.

`patience : int = 10`

Number of consecutive convergences before declaring convergence.

`rtol : float = 1e-10`

See `numpy.isclose` for details.

`atol : float = 1e-20`

See `numpy.isclose` for details.

`equal_nan : bool = False`

See `numpy.isclose` for details.

`log_iter : bool = False`

If true, log convergence message when achieved.

call

```
__call__(
    self : Any,
```

```
    state : numpy.ndarray[tuple[typing.Any, ...], numpy.dtype[numpy.float64]]
    ) -> bool:
```

Check if all variables have stabilized at current iteration.

Parameters

```
state : numpy.ndarray[tuple[typing.Any, ...], numpy.dtype[numpy.float64]]
```

Current solution state to be checked for convergence.

n_iterations

```
n_iterations(self : Any): -> int:
```

Provides access to number of iterations performed.

3.3.1.3 ComposedStabilizedConvergence

ComposedStabilizedConvergence

```
ComposedStabilizedConvergence(
    n_arrs : int,
    **kwargs : Any
    ):
```

Wrapper for checking stabilization of several arrays.
See `StabilizeNvarsConvergenceCheck` for keyword arguments; these are shared by all tested arrays. It is always possible to compose your own convergence checker using individual instances for more control over setup.

Parameters

```
n_arrs : int
```

Number of arrays to be checked for convergence.

```
kwargs :  = None
```

See `StabilizeNvarsConvergenceCheck` for details.

call

```
__call__(
    self : Any,
    *arrs : tuple[numpy.ndarray[tuple[typing.Any, ...], numpy.dtype[numpy.float64]], ...]
    ) -> bool:
```

Check if all arrays have stabilized at current iteration.

Parameters

```
*arrs : tuple[numpy.ndarray[tuple[typing.Any, ...], numpy.dtype[numpy.float64]], ...]
```

Arrays to be checked for convergence.

n_iterations

```
n_iterations(self : Any): -> int:
```

Provides access to number of iterations performed.

3.4 Symbolic functions

This module contains utilities for working with symbolic functions, especially in the context of engineering problems treated by Majordome models. It is mostly a wrapper around CasADi, organizing symbolic operations and providing additional functionality for function manipulation in a physical context.

3.4.1 PiecewiseFunction

A major issue when working with symbolic expressions and real-world data is that they do not support flow-control constructs like `if` statements. This is a problem when we want to represent functions that are defined piecewise, especially when algorithmic differentiation is involved.

As an example, take the parameterization of thermodynamic properties of chemical species. For quantitative purposes, to ensure high accuracy, researchers have found that it is best to use different polynomial fits for different temperature ranges. This means that the thermodynamic properties of a species are defined as piecewise functions of temperature, which is a problem when we want to use them in a symbolic context.

To tackle this issue in *certain* contexts, `PiecewiseSymbolicFunction` provides an interpolation between different ranges of the domain of a function. Internally it makes use of Heaviside functions to ensure that the function is continuous. This is not a perfect solution, as it does not guarantee differentiability at the breakpoints, but it is a good compromise between accuracy and simplicity. It is also worth noting that this class is not meant to be used in a general context, but rather in specific contexts where the function is known to be well-behaved.

PiecewiseSymbolicFunction

```
PiecewiseSymbolicFunction(
    breakpoints : list[float],
    functions : list[typing.Any]
    ) -> None:
```

Compose a symbolic piecewise function with CasADi.

Parameters

`breakpoints : list[float]`

List of breakpoints where the function changes.

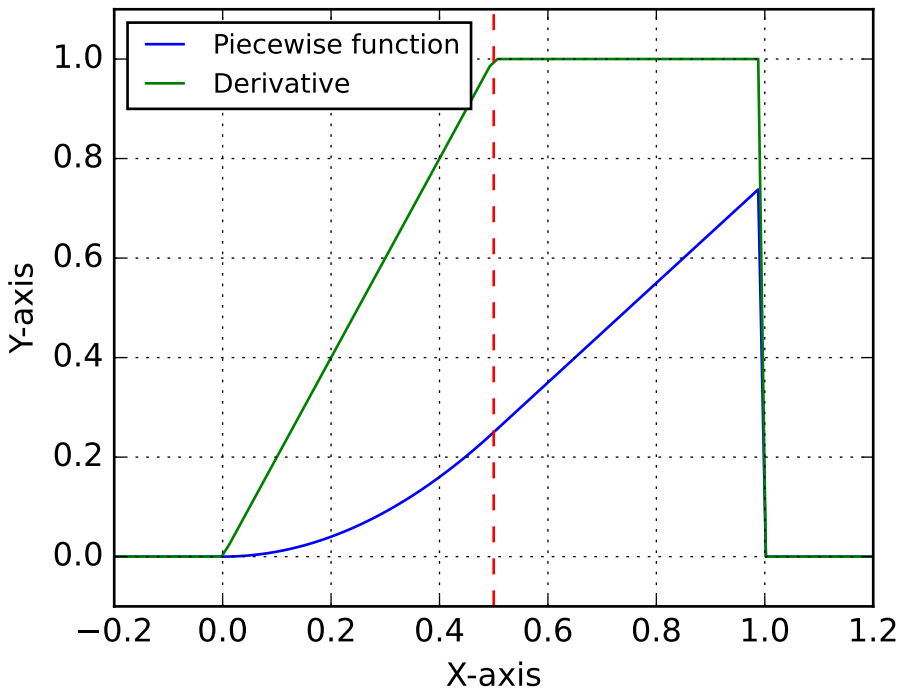
`functions : list[typing.Any]`

List of functions to apply between breakpoints. The number of functions must be one less than the number of breakpoints.

Let's illustrate its use with a simple example. We define two functions `f` and `g`, and we want to create a piecewise function that is equal to `f` for `x < 0.5` and equal to `g` for `x >= 0.5`. Functions have being created such that they evaluate to the same value at the breakpoint, as the practical case that led to the class development, *i.e.* NASA-parameterization of thermochemical properties. Below we show how to create the piecewise function, evaluate it numerically and symbolically, and compute its derivative symbolically.

```
# Define two simple functions:
f = lambda x: x**2
g = lambda x: f(0.5) + (x - 0.5)

# Create the piecewise function:
F = PiecewiseSymbolicFunction([0, 0.5, 1], [f, g])

# Evaluate numerically:
x = np.linspace(-0.2, 1.2, 100)
y = F(x)

# Evaluate symbolically:
T = SX.sym("T")
Y = F(T)

# Create a function for evaluating the derivative:
ydot = Function("fdot", [T], [jacobian(Y, T)])
```

Below we plot the piecewise function and its derivative. The function is continuous, but the derivative is not, as expected, although the transition is handled smoothly by the Heaviside functions. The red dashed line indicates the breakpoint at `x = 0.5`, where the function transitions from `f` to `g`.



3.4.2 Species thermodynamics

symbolic_thermo_factory

```
symbolic_thermo_factory(
    species : Species,
    T : SX
    ) -> AbstractSymbolicThermo:
```

Create an `AbstractSymbolicThermo` object.

Parameters

`species : Species`

Cantera species object, with NASA7 thermodynamic data.

`T : SX`

Temperature variable (symbolic).

Suppose you need to create a symbolic representation of the NASA7 thermodynamic parameterization of molecular nitrogen, species of index 47 in the GRI-Mech 3.0 mechanism shipped with Cantera. Let's start by loading and retrieving the thermodynamic data for this species.

```
gas = ct.Solution("gri30.yaml")
```

```
species = gas.species()[47]
thermo = species.thermo
```

Below we instantiate `SymbolicThermo` with the input data of the species. It is important to emphasize here that when building more complex systems, generally one should share the symbolic variables, *e.g.* temperature, across the different components of the system. For this reason, the constructor of symbolic thermodynamic classes expect the temperature symbol to be provided, instead of trying to create it internally; this is a design choice that will improve code robustness when handling mixtures and avoid many of the pitfalls of symbolic programming.

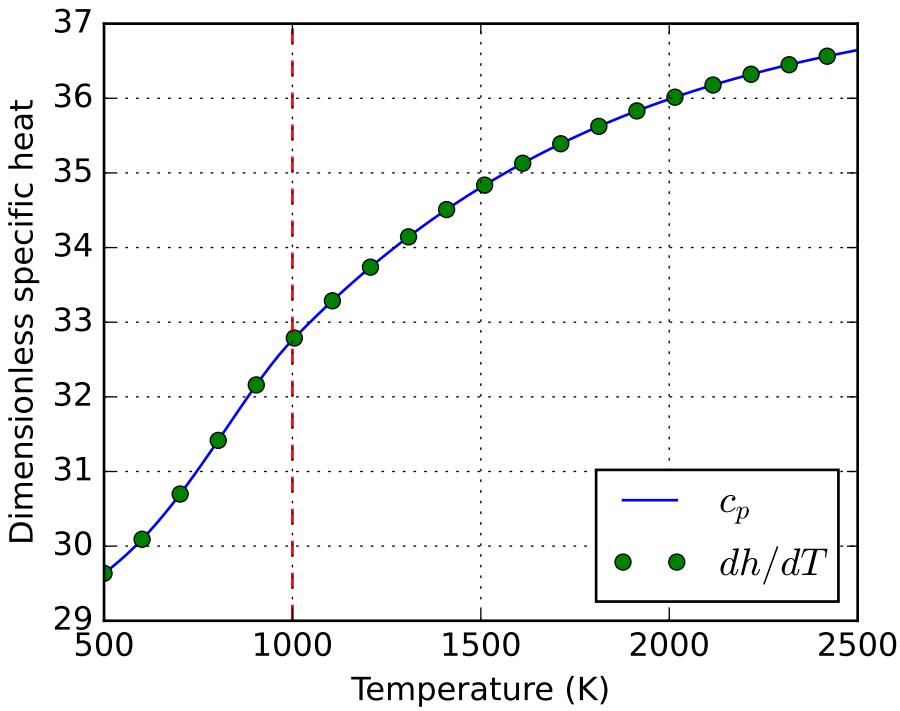
```
T = SX.sym("T")
nasa7 = symbolic_thermo_factory(species, T)
```

> **Practical usage**
>
> You could instead directly instantiate `Nasa7Thermo` with the input data, but using the factory method `SymbolicThermo.from_species` is a better choice, as it will allow you to easily switch to a different thermodynamic model. Furthermore, it is the safe choice for loading entire databases.
>
> ```
> nasa7 = Nasa7Thermo(T, species.thermo.input_data)
>
> # This is just syntactic sugar for the above:
> nasa7 = Nasa7Thermo.from_species(species, T)
> ```

Here we illustrate the use of algorithmic differentiation to compute the derivative of the specific enthalpy with respect to temperature.

```
hdot = Function("hdot", [T], [jacobian(nasa7.h(T), T)])
```

The above derivative is then confronted to the specific heat below:



Finally, we compare the deviations between symbolic and numerical (Cantera) evaluation of the thermodynamic functions. This is not for error analysis, but a proof of correctness of the symbolic implementation.

```
# XXX: Cantera evaluates quantities per kmol! Divide by 1000 to get
# per mol, as the symbolic implementation does.
c_ct = np.array([0.001 * species.thermo.cp(T) for T in T_num])
h_ct = np.array([0.001 * species.thermo.h(T) for T in T_num])
s_ct = np.array([0.001 * species.thermo.s(T) for T in T_num])

error_c = np.mean(np.abs(nasa7._cp(T_num) - c_ct))
error_h = np.mean(np.abs(nasa7._h(T_num) - h_ct))
error_s = np.mean(np.abs(nasa7._s(T_num) - s_ct))

error_c, error_h, error_s
```

```
(np.float64(4.334310688136611e-15),
 np.float64(6.311893230304122e-12),
 np.float64(3.2400748750660566e-14))
```

3.4.2.1 Nasa7Thermo

Nasa7Thermo

```
Nasa7Thermo(
    T : SX,
    input_data : dict[str, typing.Any]
    ) -> None:
```

NASA7 thermodynamic parameterization.
This class does not implement Horner polynomial evaluation, as the main use case is to create symbolic expressions that are then evaluated by CasADi, which can handle polynomial evaluation efficiently. This is intentional and allows for easy verification.
It aims at providing a similar interface as `SpeciesThermo` from Cantera, from which it retrieves the data. That means, molar properties are provided through `cp`, `h`, and `s` properties.

Parameters

`T : SX`

Temperature variable (symbolic).

`input_data : dict[str, typing.Any]`

NASA7 thermodynamic data, as provided by Cantera's `SpeciesThermo` property `input_data`.

specific_heat

```
specific_heat(
    T : Any,
    a : list[float],
    symbolic : bool = False
    ) -> casadi.casadi.Function | casadi.casadi.SX:
```

Compose NASA7 specific heat parameterization.

Parameters

`T : Any`

Temperature variable (symbolic or numeric).

`a : list[float]`

List of 7 NASA7 coefficients.

`symbolic : bool = False`

Whether to return a symbolic expression.

enthalpy

```
enthalpy(
    T : Any,
    a : list[float],
    symbolic : bool = False
    ) -> casadi.casadi.Function | casadi.casadi.SX:
```

Compose NASA7 specific enthalpy parameterization.

Parameters

`T : Any`

Temperature variable (symbolic or numeric).

`a : list[float]`

List of 7 NASA7 coefficients.

```
symbolic : bool = False
```

Whether to return a symbolic expression.

entropy

```
entropy(
    T : Any,
    a : list[float],
    symbolic : bool = False
    ) -> casadi.casadi.Function | casadi.casadi.SX:
```

Compose NASA7 specific entropy parameterization.

Parameters

```
T : Any
```

Temperature variable (symbolic or numeric).

```
a : list[float]
```

List of 7 NASA7 coefficients.

```
symbolic : bool = False
```

Whether to return a symbolic expression.

compose

```
compose(
    cls : Any,
    name : str,
    T : casadi.casadi.SX | numpy.ndarray[tuple[typing.Any, ...], numpy.dtype[numpy.float64
    data : list[list[float]],
    symbolic : bool = False
    ) -> list[casadi.casadi.Function | casadi.casadi.SX | numpy.ndarray[tuple[typing.Any,
```

Compose a list of NASA7 functions for given data.

Parameters

```
name : str
```

Name of the function ("specific_heat", "enthalpy", or "entropy", *i.e* the static methods of this class).

```
T : casadi.casadi.SX | numpy.ndarray[tuple[typing.Any, ...], numpy.dtype[numpy.float64]] |
```

Temperature variable (symbolic).

```
data : list[list[float]]
```

List of NASA7 coefficient sets for each temperature range.

```
symbolic : bool = False
```

Whether to return symbolic expressions or CasADi functions. Only relevant if `T` is symbolic.

from_species

```
from_species(
    cls : Any,
    species : Species,
    T : SX
    ) -> typing.Self:
```

Create a `Nasa7Thermo` object from a Cantera species.

Parameters

```
species : Species
```

Cantera species object, with NASA7 thermodynamic data.

```
T : SX
```

Temperature variable (symbolic).

3.4.3 Species transport

symbolic_transport_factory

```
symbolic_transport_factory(
    species : Species,
    T : SX
    ) -> AbstractSymbolicTransport:
```

Create an `AbstractSymbolicTransport` object.

Parameters

`species : Species`

Cantera species object, with transport data.

`T : SX`

Temperature variable (symbolic).

3.4.4 Kinetics parameterization

3.4.5 Solution properties

3.5 Transport properties

3.5.1 EffectiveThermalConductivity

In some fields of research, such as porous media or composite materials, the evaluation of effective thermal properties are key for simulation of macroscopic application cases. Class `EffectiveThermalConductivity` implements (static) thermal conductivity models for this sort of applications, including:

- Maxwell-Garnett approximation (`EffectiveThermalConductivity.maxwell_garnett`) as exposed in (Hanein, Glasser, and Bannerman 2017); further discussion of its origin and applicability is provided by (Kiradjiev et al. 2019).
- For porous media (packed bed in the context) with enhanced radiative effects the model by (Singh and Kaviany 1994) as discussed by (Kee et al. 2017) is implemented (`EffectiveThermalConductivity.singh1994`).

Its name is quite long, let's start by getting an alias before evaluating the desired models:

```
etc = EffectiveThermalConductivity()
```

Using Maxwell approximation, we could estimate the the effective thermal conductivity of a packed bed of particles in a *matrix* of air; assume particles loosly embeded in air and the following properties; the computed effective thermal conductivity is shown to approach the air limit:

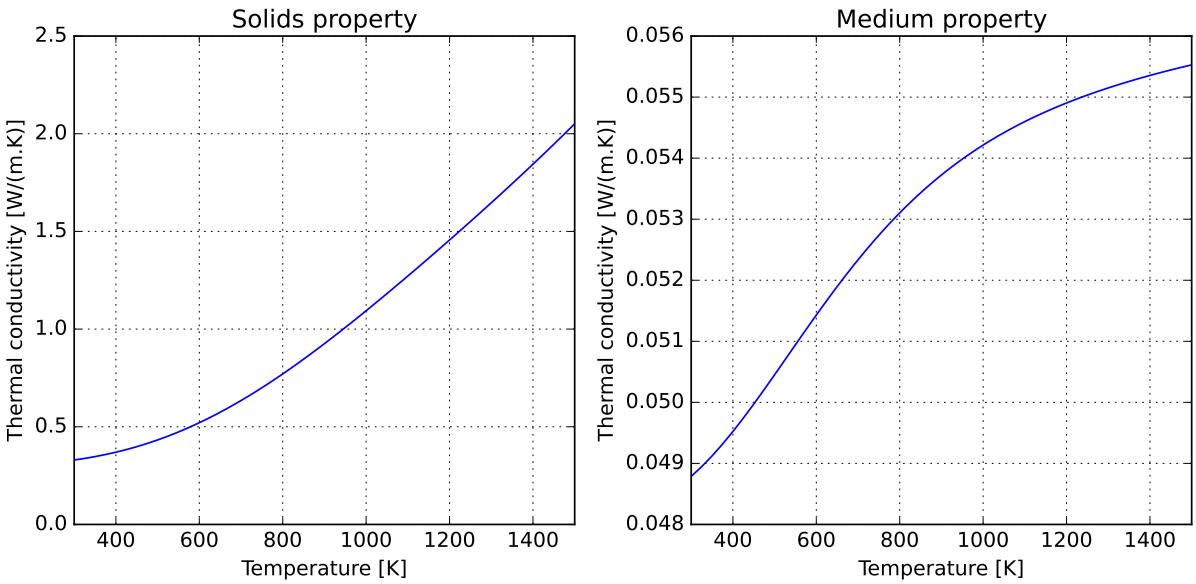
```
phi = 0.30   # Loosely packed solids [30%v]
k_g = 0.025  # Air thermal conductivity [W/(m.K)]
k_s = 1.000  # Solids thermal conductivity [W/(m.K)]

etc.maxwell_garnett(phi, k_g, k_s)
```

```
0.05396039603960396
```

For the limit of high temperatures, it is usually important to account for particle-particle radiation heat transfer; this introduces a T^3 dependence on temperature, as one should expect by linearizing Stefan-Boltzmann law.

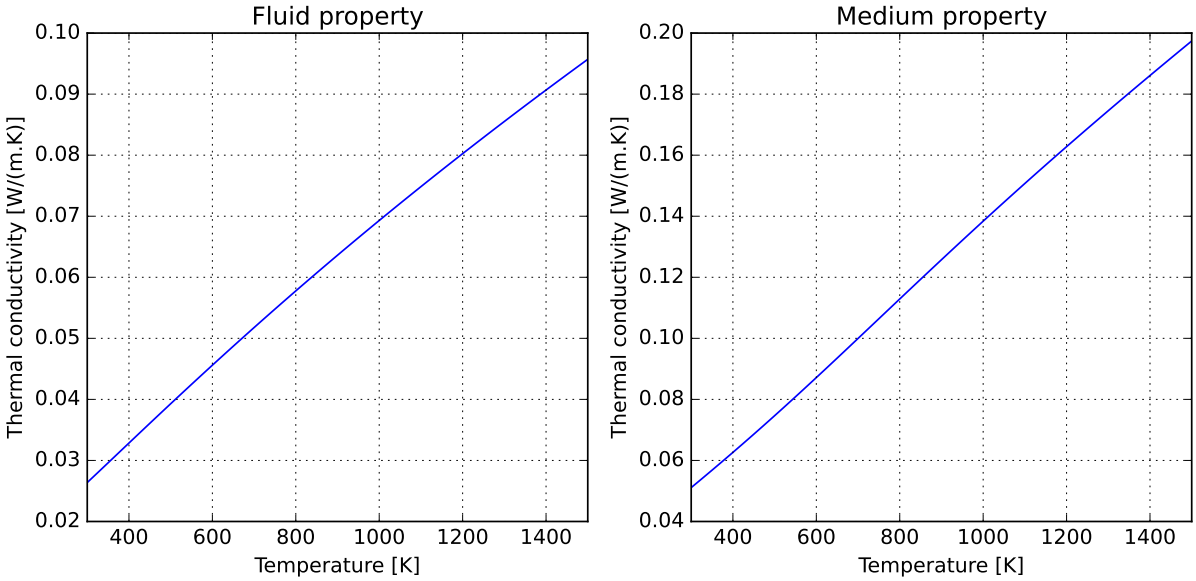
Please notice that these models compute different things; while Maxwell approximation computes the medium properties (to approximate matrix-inclusion as a single domain), Singh's model accounts only for solids properties. One might wish to combine them (warning: unverified validity!) to evaluate overall medium thermal conductivity. See the references in the class documentation for further discussion, specially the extension proposed by Kiradjiev (2019), which leads to a result similar to the assymptotic behavior displayed below.

```
d_p = 0.005
eps = 0.9

T = np.linspace(300, 1500, 50)
k_eff_s = etc.singh1994(T, phi, d_p, k_s, eps)
k_eff_m = etc.maxwell_garnett(phi, k_g, k_eff_s)

plot_etc((T, k_eff_s, k_eff_m)).resize(10, 5)
```



Due to kinetic theory implications, gas thermal conductivity tend to have a positive slope in temperature; the following illustrates how this can be accounted for and the roughly linear behaviour introduced when computing medium properties within air.

```
gas = ct.Solution("airish.yaml")
sol = ct.SolutionArray(gas, (T.shape[0],))
sol.TP = T, None

k_gas = sol.thermal_conductivity
k_eff_m = etc.maxwell_garnett(phi, k_gas, k_eff_s)

plot_etc((T, k_gas, k_eff_m)).resize(10, 5)
```



3.5.2 SolutionDimless

A dimensionless numbers calculator is provided for gas flows; it currently has a certain number of groups which are all evaluated by definition (which might change according to your field, please check the docs). The mechanics of using the class can be resumed to:

1. loading a mechanism file (Cantera YAML)
2. setting the state of the solution
3. evaluating the required properties
4. (optional) displaying a report

The meaning of the tuple of arguments provided to `set_state` is specified by `tuple_name="TPX"`, which defaults to temperature, pressure, and molar proportions. Any triplet allowed by Cantera can be specified here.

```
Tw = 1000.0  # Wall temperature [K]
U = 10.0     # Characteristic velocity [m/s]
D = 0.05     # Pipe diameter [m]
L = 1.0      # Pipe length [m]

calculator = SolutionDimless("airish.yaml")
calculator.set_state(300.0, 101325.0, "N2: 1", tuple_name="TPX")

Re   = calculator.reynolds(U, D)
Pr   = calculator.prandtl()
Sc   = calculator.schmidt()
print(calculator.report())
```

```
--------  ------------  ---------------
Reynolds  31460.9       U=10.0, L=0.05
Prandtl       0.709327
Schmidt       0.761752  mix_diff_coeffs
--------  ------------  ---------------
```

If you prefer to have direct access to the internal solution, you can set properties as usual in Cantera, but you need to keep in mind to call `update()` to refresh the internal state of the calculator. Every time the properties are updated, the internal buffer of computed dimensionless numbers is refreshed, as you migth notice in the following table.

```
calculator.solution.TPX = 300.0, 101325.0, "N2: 1"
calculator.update()

Pe_m = calculator.peclet_mass(U, L)
Pe_h = calculator.peclet_heat(U, L)
Gr   = calculator.grashof(Tw, D)
Ra   = calculator.rayleigh(Tw, D)
print(calculator.report())
```

```
-------------  ----------------  ------------------------------
Péclet (mass)  479308            U=10.0, L=1.0, mix_diff_coeffs
Péclet (heat)  446321            U=10.0, L=1.0
Grashof             1.13242e+07  Tw=1000.0, H=0.05, g=9.80665
Rayleigh            8.03259e+06  Tw=1000.0, H=0.05, g=9.80665
-------------  ----------------  ------------------------------
```

3.5.3 SutherlandFitting

To the author's knowledge, there is no standard tool to convert Cantera transport data to Sutherland parameters for use with OpenFOAM, what led to the motivation to develop `SutherlandFitting`. This simple class wraps a Cantera solution object, which it makes use for fitting Sutherland parameters to export as a table (to be used elswhere), and also allows for retrieving a converted database in OpenFOAM compatible format. The following example should be self-explanatory:

```
T = np.linspace(500, 2500, 100)

sutherland = SutherlandFitting("airish.yaml")
sutherland.fit(T, species_names=["O2", "N2"])

coef = sutherland.coefs_table
coef
```

	species	As [uPa.s]	Ts [K]	RMSE [uPa.s]
0	O2	1.873876	229.321211	0.527484
1	N2	1.619350	226.141806	0.470152

If needed, it is also possible to have direct access to the viscosity data used in parameter fitting:

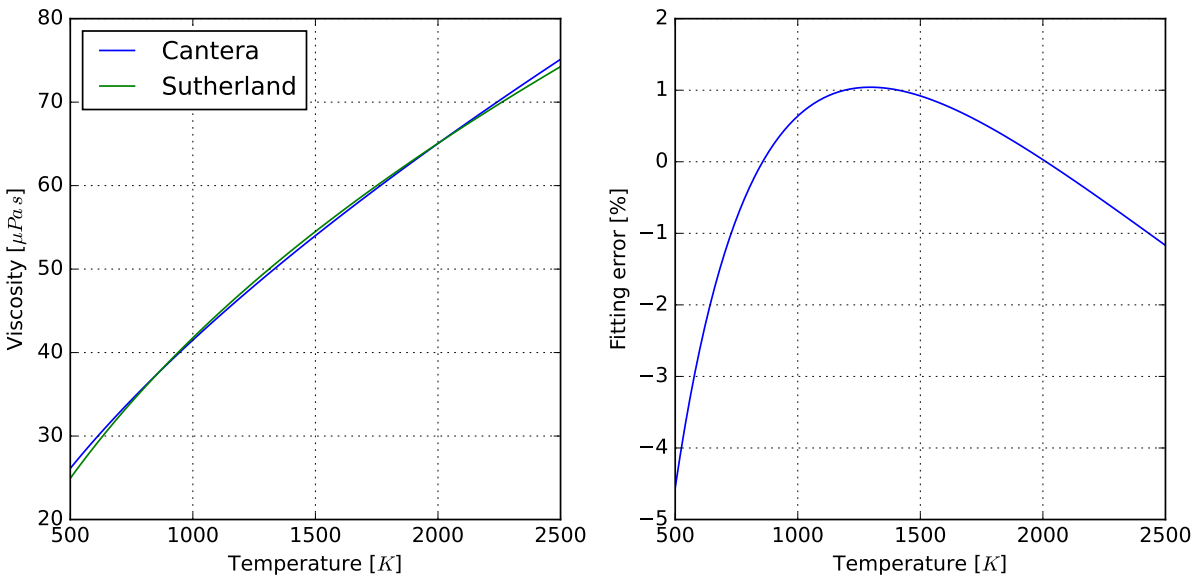
```
sutherland.viscosity.head()
```

	T	O2	N2
0	500.000000	30.045456	26.123142
1	520.202020	30.886884	26.845049
2	540.404040	31.713820	27.554780

	T	O2	N2
3	560.606061	32.527146	28.253075
4	580.808081	33.327662	28.940602

Because RMSE compresses all the error in a single value, you can check graphically where the deviations happen in the interval:

```
plot = sutherland.plot_species("N2")
```



Finally, it *also* responds to its initial goal of exporting values in OpenFOAM format:

```
print(sutherland.to_openfoam())
```

```
/* --- RMSE 0.5274842993902649 ---*/
"O2"
{
    transport
    {
        As  1.8738763185e-06;
        Ts  2.2932121082e+02;
    }
}
/* --- RMSE 0.4701517988286822 ---*/
"N2"
{
    transport
    {
        As  1.6193495479e-06;
        Ts  2.2614180620e+02;
    }
}
```

3.5.4 WSGGRadlibBordbar2020

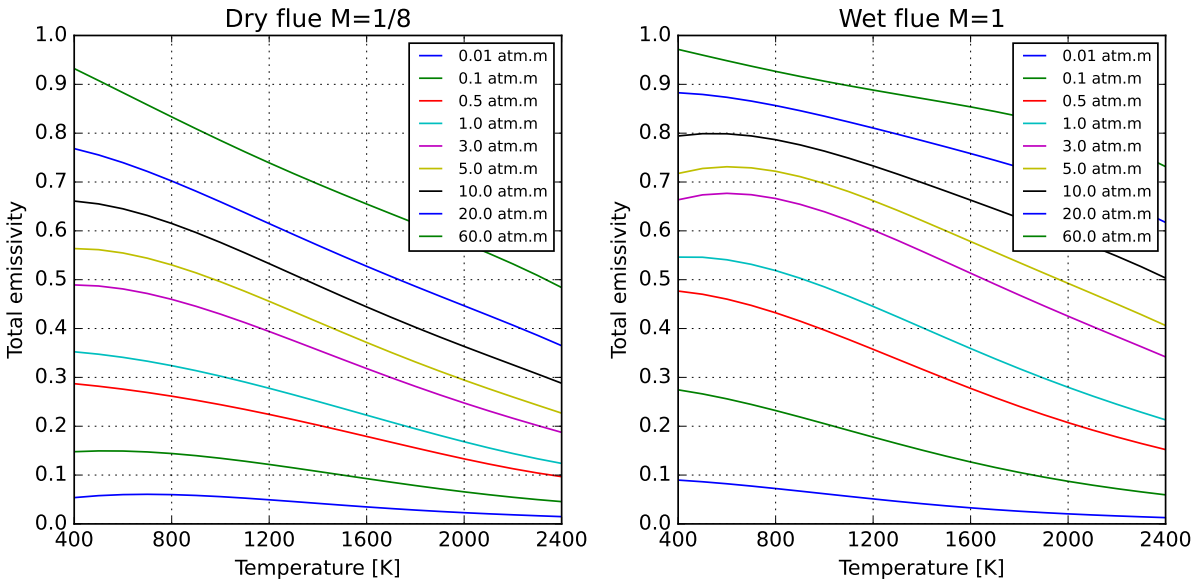
Radiative properties of gases are often required when participating media characteristics imply so. Here we implement the WSGG model of Sadeghi et al. (2021) in class `WSGGRadlibBordbar2020`. The basic usage of the model to estimate total emissivity is done as follows:

```
model = WSGGRadlibBordbar2020()
model(L=1, T=1000, P=101325, x_h2o=0.18, x_co2=0.08, fvsoot=0.0)
```

```
np.float64(0.18368577452587137)
```

3.5.4.1 Validation of Bordbar WSGG implementation

Evaluation of the model against original source of Bordbar (2014) is satisfactory, as follows:



Absorption coefficients for pure substances only reproduce approximately values reported by Bordbar (2020).

```
model(L=1, T=300, P=101325, x_h2o=0, x_co2=1)
model.absorption_coefs[1:]
```

```
array([3.388079e-02, 4.544269e-01, 4.680226e+00, 1.038439e+02])
```

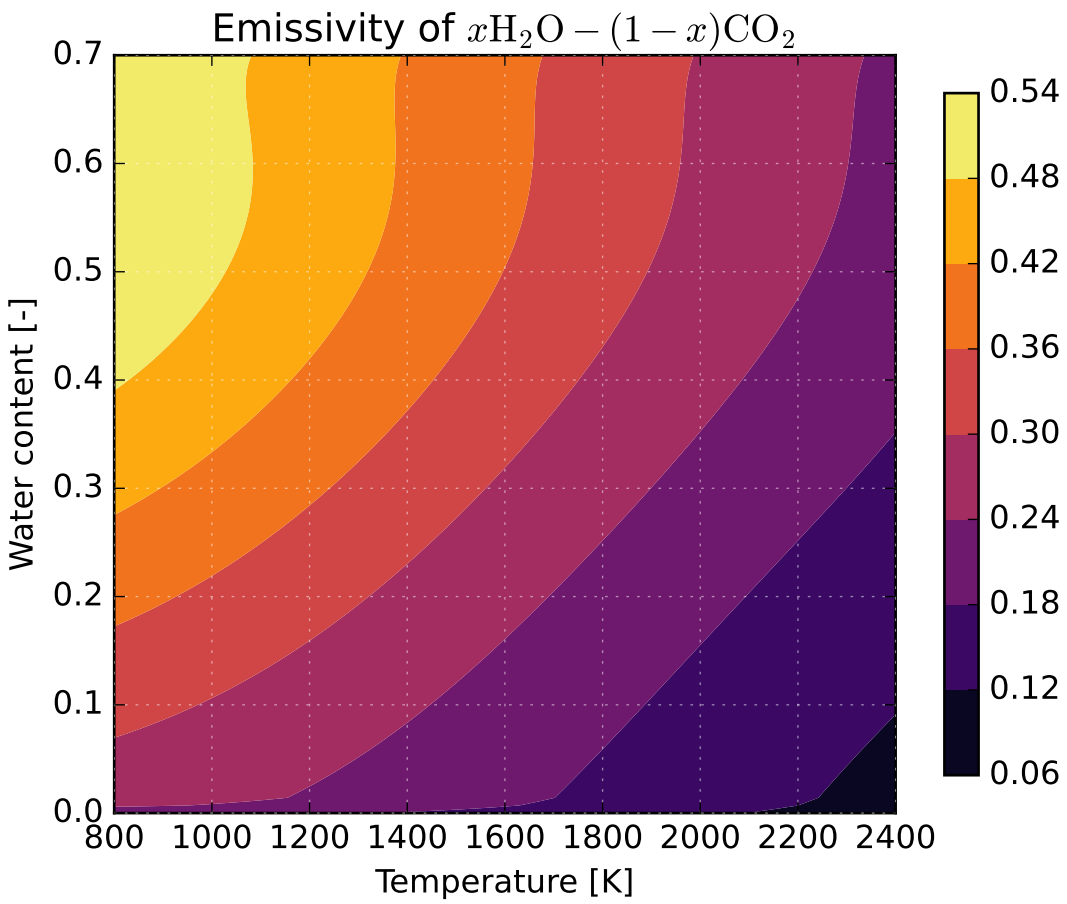
```
model(L=1, T=300, P=101325, x_h2o=1, x_co2=0)
model.absorption_coefs[1:]
```

```
array([ 0.07703541,  0.8242941 ,  6.854761  , 65.93653   ])
```

3.5.4.2 Application to radiative combustion

Below we illustrate the application of `WSGGRadlibBordbar2020` to predict the emissivity of $x\mathrm{H_2O} - (1-x)\mathrm{CO_2}$ mixtures over a broad temperature range applicable to the analysis of combustion processes. One observes an important participation of flue gases, especially at lower temperatures.

```
wsgg = WSGGRadlibBordbar2020()

@np.vectorize
def emissivity(T, X, L=1, P=101325):
    return wsgg(L=L, T=T, P=P, x_h2o=X, x_co2=1-X, fvsoot=0.0)

T = np.linspace(800, 2400, 100)
X = np.linspace(0, 0.7, 100)

T, X = np.meshgrid(T, X)
eps = emissivity(T, X)
plot_emissivity(T, X, eps)
```



3.6 Computer vision

Upcoming...

4 Simulation

aka majordome.simulation

4.1 Elmer Multiphysics

4.1.1 TimeStepAccumulator

TimeStepAccumulator

```
TimeStepAccumulator(*time_steps : ConstantTimeStepInterval): -> None:
```

Helper for creating cumulative time steps in Elmer SIF files.

Parameters

`*time_steps : ConstantTimeStepInterval`

Variable number of time step intervals to accumulate.

add_block

```
add_block(
    self : Any,
    *,
    duration : float | None = None,
    end_time : float | None = None,
    **kws : Any
    ) -> None:
```

Add a new block of time steps with the given parameters.

Parameters

`duration : float | None = None`

Duration of the new time step block. If not provided, then `end_time` must be provided.

`end_time : float | None = None`

End time of the new time step block. If not provided, then `duration` must be provided. If provided, the duration of the new block will be calculated as `end_time - self.duration`.

`kws : = None`

Additional keyword arguments to pass to the `ConstantTimeStepInterval` constructor.

append

```
append(
    self : Any,
    time_step : ConstantTimeStepInterval
    ) -> None:
```

Append a new time step to the accumulator.

Parameters

`time_step : ConstantTimeStepInterval`

The time step interval to append to the accumulator.

to_dataframe

```
to_dataframe(self : Any): -> DataFrame:
```

Convert the accumulated time steps into a table.

In general the usage of the `TimeStepAccumulator` is as follows:

```
! >> Total duration: 0.001 << !
! Step Duration(3)      =         1e-05         9e-05        0.0009
Timestep Sizes(3)       =  1.000000e-06  1.000000e-05  1.000000e-04
Timestep Intervals(3)   =            10             9             9
Output Intervals(3)     =            10             9             9
```

4.1.1.1 ConstantTimeStepInterval

In some special cases, when using `TimeStepAccumulator` you might want to add time steps one by one, which can be done using the `append` method in combination with the `ConstantTimeStepInterval` described below.

ConstantTimeStepInterval

```
ConstantTimeStepInterval(
    duration : float,
```

```
    time_step : float | None = None,
    save_interval : float | None = None,
    *,
    min_steps : int = 2
    ) -> None:
```

Create a constant time step interval for a simulation.

Parameters

`duration : float`

Total duration of the time step interval.

`time_step : float | None = None`

Size of each time step. If not provided, it will be calculated based on the `duration` and `min_steps` parameters.

`save_interval : float | None = None`

Interval at which to save results. If not provided, it will be set equal to `time_step`.

`min_steps : int = 2`

Minimum number of time steps to use if `time_step` is not provided. Must be at least 1. Default is 2.

into_formatted_data

```
into_formatted_data(self : Any): -> list[str]:
```

Format the time step data for output in an Elmer SIF file.

4.1.2 ElmerConvergenceData

ElmerConvergenceData

```
ElmerConvergenceData(file_path : str | pathlib.Path): -> None:
```

Class for handling convergence data from Elmer simulations.

Parameters

```
file_path : str | pathlib.Path
```

Path to the file containing convergence data, typically output by Elmer's SaveLine solver.

get_solver_data

```
get_solver_data(
    self : Any,
    solver_id : int
    ) -> DataFrame:
```

Get convergence data for a specific solver.

Parameters

```
solver_id : int
```

The ID of the solver to filter by.

plot_solver_convergence

```
plot_solver_convergence(
    self : Any,
    *,
    solver_id : int,
    final_iter : bool = False,
    time_axis : numpy.ndarray[tuple[typing.Any, ...], numpy.dtype[~_ScalarT]] | None = Non
    plot : majordome.utilities.MajordomePlot | None = None
    ) -> MajordomePlot:
```

Plot convergence data for a specific solver.

Parameters

```
solver_id : int
```

The ID of the solver to plot.

```
final_iter : bool = False
```

If True, only plot the final iteration for each timestep.

```
time_axis : numpy.ndarray[tuple[typing.Any, ...], numpy.dtype[~_ScalarT]] | None = None
```

Optional array of time values to use as the x-axis. If not provided, the timestep numbers will be used.

```
plot : majordome.utilities.MajordomePlot | None = None
```

Placeholder for plot object provided by decorator.

tracked_solvers

```
tracked_solvers(self : Any): -> list[int]:
```

List of unique solvers tracked in the data.

4.1.3 ElmerTabularData

ElmerTabularData

```
ElmerTabularData(
    fname : str | pathlib.Path,
    *,
    fmeta : str | pathlib.Path | None = None
    ):
```

Class to represent the data of a SaveLine results file.

Parameters

```
fname : str | pathlib.Path
```

The path to the data file output of Elmer's SaveLine solver.

```
fmeta : str | pathlib.Path | None = None
```

The path to the associated metadata file. If not provided, the class will attempt to find a metadata file in the same directory as the data file, with the same name but with a .names extension. If still not found, it will look for the metadata file one level above the data file directory.

metadata

```
metadata(self : Any): -> ElmerTabularMetadata:
```

Provides access to the metadata of the save line data.

data

```
data(self : Any): -> DataFrame:
```

Provides access to the data of the save line.

4.1.4 ElmerTabularMetadata

ElmerTabularMetadata

```
ElmerTabularMetadata(filepath : str | pathlib.Path)::
```

Class to hold metadata from Elmer SaveLine .names file.

Parameters

```
filepath : str | pathlib.Path
```

Path to the .names metadata file.

metadata

```
metadata(self : Any): -> dict:
```

Return the metadata as a dictionary.

columns

```
columns(self : Any): -> dict:
```

Return the column definitions as a dictionary.

4.2 Ansys Fluent

4.2.1 FluentFvParticlesParser

This class provides functionality for parsing particle track files. Fluent export DPM results in several formats; `majordome` supports *fieldview* particle tracks through `FluentFvParticlesParser`. Loading a file is as simple as:

You can retrieve the names of columns and number of individual tracks with the following properties:

```
(['x', 'y', 'z', 'residence_time', 'time_step'], 2)
```

For convenience, tracks can be recovered as data frames through object indexing:

	x	y	z	residence_time	time_step
0	0.128501	0.537214	-0.622358	0.000000	0.000000e+00
1	0.128493	0.537929	-0.619682	0.000107	1.697420e-07
2	0.128493	0.537964	-0.619554	0.000112	2.738057e-07
3	0.128488	0.538534	-0.617419	0.000176	4.008972e-07
4	0.128477	0.539842	-0.612481	0.000299	7.041471e-07

4.2.2 FluentInterpolationParser

Sometimes is is also useful to parse interpolation files. It enables one to compute new quantites for an interpolation files may be useful, especially in reasearch settings. Class `FluentInterpolationParser` is able to read the contents of an interpolation file exported in plain text format. Below you find an example of parsing.

```
            x0 in range (        11:        16)
            x1 in range (        17:        22)
      pressure in range (        23:        28)
    x-velocity in range (        29:        34)
    y-velocity in range (        35:        40)
   temperature in range (        41:        46)
             k in range (        47:        52)
         omega in range (        53:        58)
```

Names of variables are provided by property `variable_names`:

```
['x0',
 'x1',
 'pressure',
 'x-velocity',
 'y-velocity',
 'temperature',
 'k',
 'omega']
```

Valid names can be used to retrieve data for a given variable:

```
array([0.1, 0.1, 0.2, 0.3, 0.4])
```

The whole data table can be retreived with `data` property, where each variable correspond to a column:

```
array([[0.1, 1. , 9. , 9. , 9. , 9. , 9. , 9. ],
       [0.1, 2. , 1. , 1. , 1. , 1. , 1. , 1. ],
       [0.2, 3. , 2. , 2. , 2. , 2. , 2. , 2. ],
       [0.3, 4. , 3. , 3. , 3. , 3. , 3. , 3. ],
       [0.4, 5. , 4. , 4. , 4. , 4. , 4. , 4. ]])
```

4.3 OpenFOAM

4.4 Meshing

4.4.1 GmshOCCModel

GmshOCCModel

```
GmshOCCModel(
    *,
    render : bool = False,
    name : str = 'domain',
    **kws : Any
    ) -> None:
```

Wrapper to manage OCC models with an OOP approach.

Parameters

`render : bool = False`

Whether to launch the Gmsh GUI after building the model.

`name : str = 'domain'`

Name of the Gmsh model.

`ks : dict = None`

Provide configuration for gmsh internals.

5 Utilities

aka majordome.utilities

This module provides a set of utilities that are whether too general purpose for integrating another module or still waiting to find their definitive home. These include constants, type aliases, and functionalities of different kinds. Regarding this last kind of components, this tutorial aims at illustrating their use for practical documentation.

5.1 Plotting

5.1.1 MajordomePlot

One might ask why a wrapper around Matplotlib's `Figure` and `Axes` is necessary, and I would strongly agree that it isn't. However, keeping all the plots in a report consistent is a huge effort, and being able to standardize the plots without having to write new themes is a nice feature. That's the main goal of `MajordomePlot`: simplifying the plotting of publication-quality figures across Majordome (and beyond!). Below we illustrate how to use this functionality to ensure all the plots in a project look the same.

MajordomePlot

```
MajordomePlot(
    shape : tuple[int, int] = (1, 1),
    style : str = 'classic',
    *,
    size : tuple[float, float] | None = None,
    xlabel : str | list[str] | None = None,
    ylabel : str | list[str] | None = None,
    opts : dict[str, typing.Any] = {'facecolor': 'white'}
    ) -> None:
```

Handles the creation and management of plots.
Provides a handle for creating and managing plots using Matplotlib. It is aimed at standardizing the plot creation process across the Majordome framework.

Parameters

`shape : tuple[int, int] = (1, 1)`

Shape of the plot as (n_rows, n_cols).

`style : str = 'classic'`

Matplotlib style to use for the plot.

`size : tuple[float, float] | None = None`

Size of the plot in inches as (width, height). If None, the default size will be used.

`xlabel : str | list[str] | None = None`

Label(s) for the x-axis. If a list is provided, each subplot will get its own label.

`ylabel : str | list[str] | None = None`

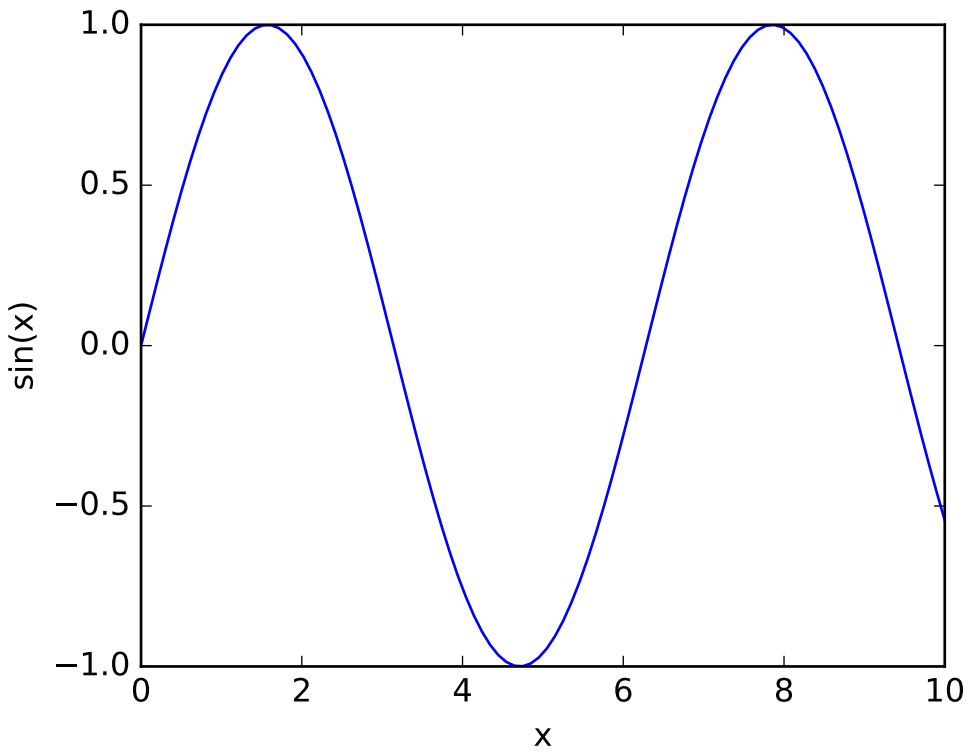
Label(s) for the y-axis. If a list is provided, each subplot will get its own label.

`opts : dict[str, typing.Any] = {'facecolor': 'white'}`

Additional options to pass to `plt.subplots()`.

Let's see how one would expect to use it from the above constructor signature. This is in fact almost the same as calling `plt.subplots()`, and that is what the constructor does under the hood. This is a *negative* example actually, the author has *never* used the constructor directly, but instead uses the `new` classmethod, which is a much more convenient way to create a plot. More on that later.

```
x = np.linspace(0, 10, 100)
mp = MajordomePlot(shape=(1, 1), size=(5, 4))
mp.axes[0].plot(x, np.sin(x))
mp.axes[0].set_xlabel("x")
mp.axes[0].set_ylabel("sin(x)")
pass
```



The story behind this class dates a few years back, when I was learning about Python decorators. As an exercise, I coded what later became the `new` classmethod, which is a decorator that creates a new `MajordomePlot` and passes it to the decorated function. This is a very convenient way to create plots, as it allows you to focus on the plotting code and not worry about the boilerplate of creating the plot and setting the labels.

new

```
new(
    cls : Any,
    _func : typing.Optional[typing.Callable[~Params, typing.Any]] = None,
    *,
    shape : tuple[int, int] = (1, 1),
    style : str = 'classic',
    sharex : bool = True,
    facecolor : str = 'white',
    grid : bool = True,
    size : tuple[float, float] | None = None,
    xlabel : str | list[str] | None = None,
    ylabel : str | list[str] | None = None
    ) -> typing.Callable[[typing.Callable[~Params, typing.Any]], typing.Any]:
```

Wraps a function for ensuring a standardized plot.

Parameters

`_func : typing.Optional[typing.Callable[~Params, typing.Any]] = None`

Function to wrap, if any.

```
shape : tuple[int, int] = (1, 1)
```

Shape of the plot as (n_rows, n_cols).

```
style : str = 'classic'
```

Matplotlib style to use for the plot. By default "classic".

```
sharex : bool = True
```

Whether to share x-axis among subplots.

```
facecolor : str = 'white'
```

Face color of the figure.

```
grid : bool = True
```

Whether to display grid lines.

```
size : tuple[float, float] | None = None
```

Size of the plot in inches as (width, height). If None, the default size will be used. By default None.

```
xlabel : str | list[str] | None = None
```

Label(s) for the x-axis. If a list is provided, each subplot will get its own label. By default None.

```
ylabel : str | list[str] | None = None
```

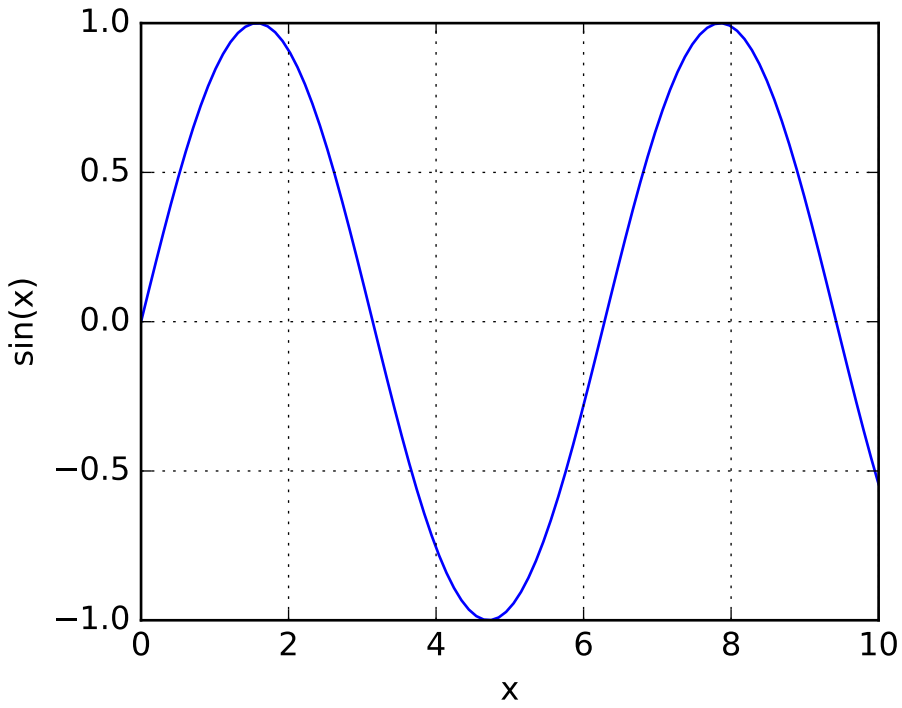
Label(s) for the y-axis. If a list is provided, each subplot will get its own label. By default None.

In the following example, we make use of this decorator to parametrize the labels of the plot. Internally, we must expect a *keyword-only* argument `plot`, which is the `MajordomePlot` instance created by the decorator, from which we can access the `Figure` and `Axes` objects as usual. The rest of the code is just standard Matplotlib code, with the exeption that the axes is raveled, so we can access it with `ax[0]` instead of `ax[0, 0]`. Returning `plot` is optional (it is done by the decorator already), but it avoids linting errors if you want to annotate the return type as `MajordomePlot`.

Here we create a function that will be used below in the examples. In summary, to get it working, you need to:

- Use decorator `@MajordomePlot.new` with all its configurable attributes; for details, please refer to its API documentation.
- Have a function with signature `func(*args, plot=None, **kwargs) -> None`, where it is recommended (for linter) to provide explictly keyword `plot=None`.
- Unpack `fig, ax = plot.subplots()` or just `_, ax = plot.subplots()`, as needed, inside the figure; these contain standard `matplotlib` figure and axes.

```
@MajordomePlot.new(size=(5, 4), xlabel="x", ylabel="sin(x)")
def plot_sin(x, *, plot, **kwargs) -> MajordomePlot:
    _, ax = plot.subplots()
    ax[0].plot(x, np.sin(x))
    return plot

plot = plot_sin(np.linspace(0, 10, 100))
```



The following elements document the members and properties of the class.

resize

```
resize(
    self : Any,
    w : float,
    h : float
    ) -> None:
```

Resize a plot with width and height in inches.

Parameters

`w : float`

Width of the plot in inches.

`h : float`

Height of the plot in inches.

savefig

```
savefig(
    self : Any,
    filename : str | pathlib.Path,
    **kwargs : Any
    ) -> None:
```

Wrapper for saving a figure to file.

Parameters

`filename : str | pathlib.Path`

Path to save the figure to.

`kwargs : None = None`

Additional keyword arguments to pass to `Figure.savefig()`.

subplots

```
subplots(self : Any): -> tuple[matplotlib.figure.Figure, list[matplotlib.axes._axes.Axes]]
```

Provides access to underlying figure and axes.

show

```
show(self : Any): -> None:
```

Display the plot.

figure

```
figure(self : Any): -> Figure:
```

Provides access to underlying figure.

axes

```
axes(self : Any): -> list[matplotlib.axes._axes.Axes]:
```

Provides access to underlying axes.

5.1.2 PowerFormatter

PowerFormatter

```
PowerFormatter(**kwargs : Any): -> None:
```

Formatter for power of ten in numerical axes.

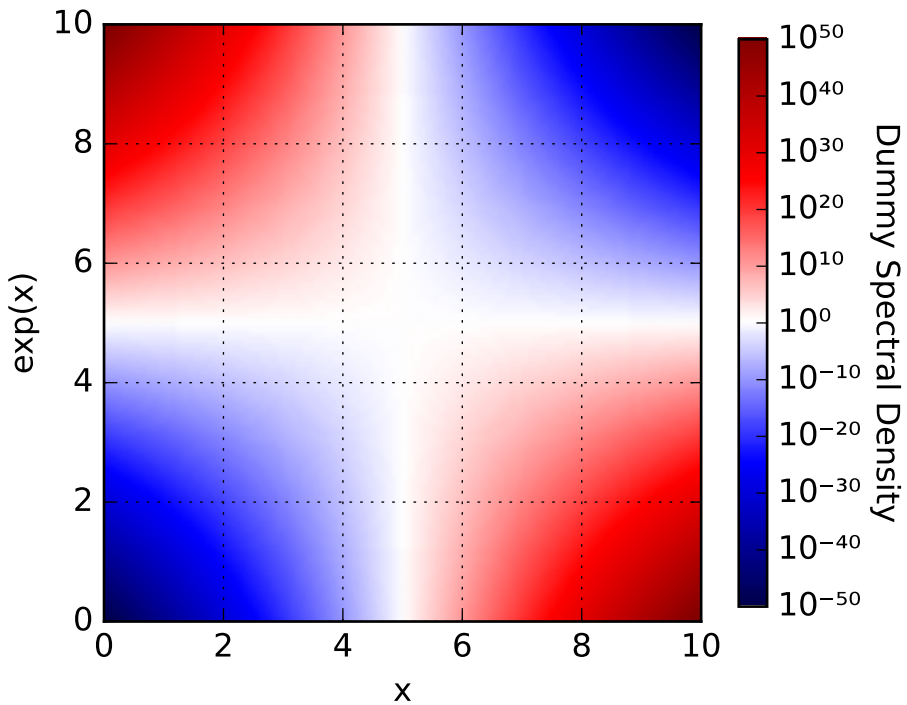
Parameters

`values : str = None`

String of characters to be replaced by their superscript counterparts. By default "0123456789-".

`supers : str = None`

String of superscript characters corresponding to `values`. By default " ¹²³ ".



5.1.3 General utilities

centered_colormap

```
centered_colormap(
    name : str,
    vmin : float,
    vmax : float,
    vcenter : float = 0.0
    ) -> LookupTable:
```

Ensure the center of a colormap is at zero.

Parameters

`name : str`

Name of the colormap to use.

`vmin : float`

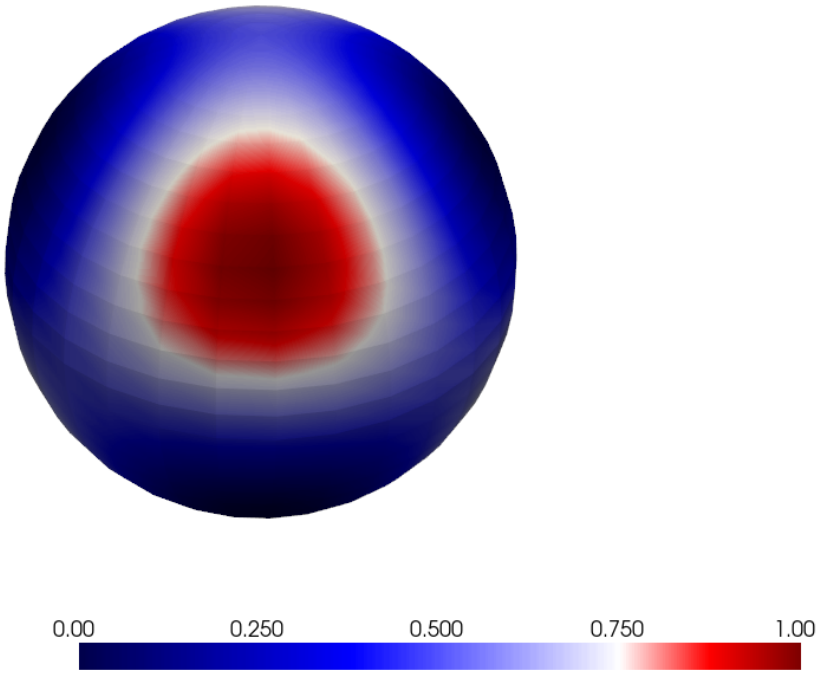
Minimum value of the data range.

`vmax : float`

Maximum value of the data range.

`vcenter : float = 0.0`

Center value of the colormap, by default 0.



5.2 LaTeX

Upcoming...

5.3 Argument parsing

5.3.1 FuncArguments

```
def f(*args, **kwargs):
    """ Arbitrary interface function for illustration. """
    f.parser.update(*args, **kwargs)

    try:
        print(f"a = {f.parser.get('a')}, b = {f.parser.get('b')}")
        f.parser.close()
    except Exception as err:
        print(err)
        pass
```

- Arguments are both positional-mandatory

```
f.parser = FuncArguments()
f.parser.add("a", index=0)
f.parser.add("b", index=1)

f(3)
f(3, 4)
```

```
Cannot retrieve mandatory positional argument at position 1
a = 3, b = 4
```

- Arguments are both keyword-only

```
f.parser = FuncArguments()
f.parser.add("a", default="foo")
f.parser.add("b", default="bar")

f(3)
f(3, 4)
f(3, b=4)
f(a=3, b=4)
```

```
a = foo, b = bar
Too many positional arguments, got 1 but only 0 were used.
a = foo, b = bar
Too many positional arguments, got 2 but only 0 were used.
a = foo, b = 4
Too many positional arguments, got 1 but only 0 were used.
a = 3, b = 4
```

- One positional and another (maybe) positional

```
f.parser = FuncArguments()
f.parser.add("a", index=0)
f.parser.add("b", index=1, default=6)

f(3)
f(3, 4)
f(3, 4, b=4)
```

```
a = 3, b = 6
a = 3, b = 4
Cannot have both positional and keyword version of b (1) simultaneously
```

- Badly configured

```
f.parser = FuncArguments()

try:
    f.parser.add("a")
    f.parser.add("b")
except Exception as err:
    print(err)
```

```
Argument must be either positional or keyword, cannot be neither: a
```

5.3.2 Creating class constructors

You should probably not do this, but it is quite fun!

```
def _init_some_class(cls):
    """ Decorator to enhance SomeClass with argument parsing. """
    orig_init = cls.__init__

    # Maybe stash for later use?
    # cls.__orig_init__ = orig_init

    parser = FuncArguments(greedy_args=False, pop_kw=True)
    parser.add("a", 0)
    parser.add("x", default=None)

    @wraps(orig_init)
    def new_init(self, *args, **kwargs):
        parser.update(*args, **kwargs)
        a = parser.get("a")
        x = parser.get("x")

        orig_init(self, *parser.args, **parser.kwargs)
        parser.close()

        # -- logic goes here
        print(f"some a = {a}")
        print(f"some x = {x}")

        return None

    cls.__init__ = update_wrapper(new_init, orig_init)
    return cls

class BaseClass:
    def __init__(self, *args, **kwargs) -> None:
        print(f"args   = {args}")
        print(f"kwargs = {kwargs}")

@_init_some_class
class SomeClass(BaseClass):
    def __init__(self, *args, **kwargs) -> None:
        super().__init__(*args, **kwargs)

some = SomeClass(1, 2, 3, x=1, y=2)
```

```
args   = (1, 2, 3)
kwargs = {'y': 2}
some a = 1
some x = 1
```

5.4 PDF Tools

5.4.1 PdfToTextConverter

Below we illustrate the usage of `PdfToTextConverter`. Please notice that data curation of extracted texts is still required if readability is a requirement. If quality of automated extractions is often poor for a specific language, you might want to search the web how to *train tesseract*, that topic is not covered here.

Note: this note assumes tesseract and poppler, and ImageMagick are available in system path. Under Windows you might struggle to get them all working together, please check Majordome's Kompanion for automatic installation.

Install dependencies on Ubuntu 22.04:

```
sudo apt install tesseract-ocr imagemagick poppler-utils
```

In case of Rocky Linux 9:

```
sudo dnf install tesseract tesseract-langpack-eng ImageMagick poppler-utils
```

Assuming the dependencies are found in the path, it is simply a matter of creating a converter:

```
converter = PdfToTextConverter()
```

For generated PDF (not scanned documents), it is much faster to avoir using OCR; below we show the metadata from a paper:

```
data = converter("data/pdftools/paper.pdf", use_ocr=False)
data.meta
```

```
{'/Author': "W. Dal'Maz Silva",
 '/CreationDate': "D:20170403224009+05'30'",
 '/Creator': 'Elsevier',
 '/CrossMarkDomains[1]': 'elsevier.com',
 '/CrossMarkDomains[2]': 'sciencedirect.com',
 '/CrossmarkDomainExclusive': 'true',
 '/CrossmarkMajorVersionDate': '2010-04-23',
 '/ElsevierWebPDFSpecifications': '6.5',
 '/Keywords': 'Hardness measurement; Martensite; Low-alloy steel; Precipitation',
 '/ModDate': "D:20170403224009+05'30'",
 '/Subject': 'Materials Science & Engineering A, 693 (2017) 225-232. doi:10.1016/j.msea.2017
 '/Title': 'Carbonitriding of low alloy steels_ Mechanical and metallurgical responses',
 '/doi': '10.1016/j.msea.2017.03.077',
 '/robots': 'noindex'}
```

For scanned documents, by default if OCR is not enabled it will be used as a fallback method for text extraction:

```
data = converter("data/pdftools/scanned.pdf", last_page=1)
data.content[:500]
```

```
'549\n\n5.. Uber die von der molekularkinetischen Theorie\nder Wirme geforderte Bewegung vor
\nkinetischen Theorie der Warme in Flissigkeiten suspendierte\nKorper von mikroskopisch sich
\nkularbewegung der Wirme Bewegungen von solcher GrifSe\nausfiihren miissen, daB diese Beweg
```

5.5 Misc

5.5.1 ColorPrint

```
This is a red message.
This is a green message.
This is a blue message.
This is a yellow message.
This is a cyan message.
```

6 Documents

aka majordome_utilities.documents

SignatureEntry

```
SignatureEntry(
    fn : type,
    *,
    greedy : bool = True
    ) -> None:
```

Builds the representation of a function signature.

Parameters

`fn : type`

The function to build the signature for.

`greedy : bool = True`

Whether to raise errors for missing docstrings or parameters.

documentation

```
documentation(
    self : Any,
    **kwargs : Any
    ) -> None:
```

Generates the markdown documentation for the function.

Parameters

`title : str = None`

The title to use for the section header and callout. By default, the function name is used.

6.1 Tutorial

```
class Dummy:
    """ A dummy class for testing purposes. """
    def method(self, a, b):
        """ This is a method.

        Parameters
        ----------
        a : int
            This is the first parameter.
        b : int
            This is the second parameter.
        """
        pass

    @staticmethod
    def static_method(a, b):
        """ This is a static method.

        Parameters
        ----------
        a : int
            This is the first parameter.
        b : int
            This is the second parameter.
        """
        pass

    @classmethod
    def class_method(cls, a, b):
        """ This is a class method.

        Parameters
        ----------
        a : int
            This is the first parameter.
        b : int
            This is the second parameter.
        """
        pass

    @property
    def prop(self) -> int:
        """ This is a property. """
        return 42
```

Documentation of all members at once is discouraged, but it is possible to do so by calling all methods of ‘`DocumentedClass`:

```
doc = DocumentedClass(Dummy)
doc.constructor()
```

```
doc.members()
doc.properties()
doc.classmethods()
doc.staticmethods()
```

For generating more organized documentation, it is possible to selectively include or exclude static methods, class methods and properties. This allows to generate separate sections for each type of member, which can be more readable and easier to navigate.

The constructor:

Dummy

```
Dummy():
```

A dummy class for testing purposes.

Only normal methods:

method

```
method(
    self : Any,
    a : Any,
    b : Any
    ):
```

This is a method.

Parameters

`a : Any`

This is the first parameter.

`b : Any`

This is the second parameter.

Only static methods:

static_method

```
static_method(
    a : Any,
    b : Any
    ):
```

This is a static method.

Parameters

`a : Any`

This is the first parameter.

`b : Any`

This is the second parameter.

Only class methods:

class_method

```
class_method(
    cls : Any,
    a : Any,
    b : Any
    ):
```

This is a class method.

Parameters

`a : Any`

This is the first parameter.

`b : Any`

This is the second parameter.

Only properties:

prop

```
prop(self : Any): -> int:
```

This is a property.

6.2 Examples

A minimal function with no parameters and no docstring:

```python
def awesome_useless():
    pass
```

awesome_useless

```
awesome_useless():
```

(missing summary)

A function with variadic parameters and no documented parameters:

```python
def awesome_useless(*args, **kwargs):
    """ Some useless function. """
    pass
```

awesome_useless

```
awesome_useless(
    *args : Any,
    **kwargs : Any
    ):
```

Some useless function.

Parameters

```
*args : Any
```

(missing description)

A function with variadic parameters with type annotation and explictly documented keyword parameter:

```python
def awesome_useless(*args: tuple[int, ...], **kwargs):
    """ Some useless function.

    Parameters
    ----------
    args : tuple
        Positional arguments. This is intended to be a catch-all for
        any number of positional arguments. Let's write a long text.
    some_kwarg : int = 1
        Keyword argument. This is intended to be a catch-all for any
        number of keyword arguments. Let's write a long text.
    """
    pass
```

awesome_useless

```
awesome_useless(
    *args : tuple[int, ...],
    **kwargs : Any
    ):
```

Some useless function.

Parameters

`*args : tuple[int, ...]`

Positional arguments. This is intended to be a catch-all for any number of positional arguments. Let's write a long text.

`some_kwarg : int  = '1'`

Keyword argument. This is intended to be a catch-all for any number of keyword arguments. Let's write a long text.

A function with a positional-only parameter, a keyword-only parameter and variadic parameters:

```
def awesome_useless(a: int, /, b, **kwargs):
    """ Some useless function. """
    pass
```

awesome_useless

```
awesome_useless(
    a : int,
    /,
    b : Any,
    **kwargs : Any
    ):
```

Some useless function.

Parameters

`a : int`

(missing description)

`b : Any`

(missing description)

Function with all kinds of parameters:

```
def awesome_useless(x, /, y: int = 0, *, z: int = 0, **kwargs) -> None:
    """ Some useless function.

    Parameters
    ----------
    x : int
        Nothing to say about this.
    y : int
        Nothing to say about this.
    z : int
        Nothing to say about this.
    xx : int
        Nothing to say about this.
    """
    pass
```

awesome_useless

```
awesome_useless(
    x : Any,
    /,
    y : int = 0,
    *,
    z : int = 0,
    **kwargs : Any
    ) -> None:
```

Some useless function.

Parameters

`x : Any`

Nothing to say about this.

`y : int = 0`

Nothing to say about this.

`z : int = 0`

Nothing to say about this.

`xx : int = None`

Nothing to say about this.

Part II

Reactor Engineering

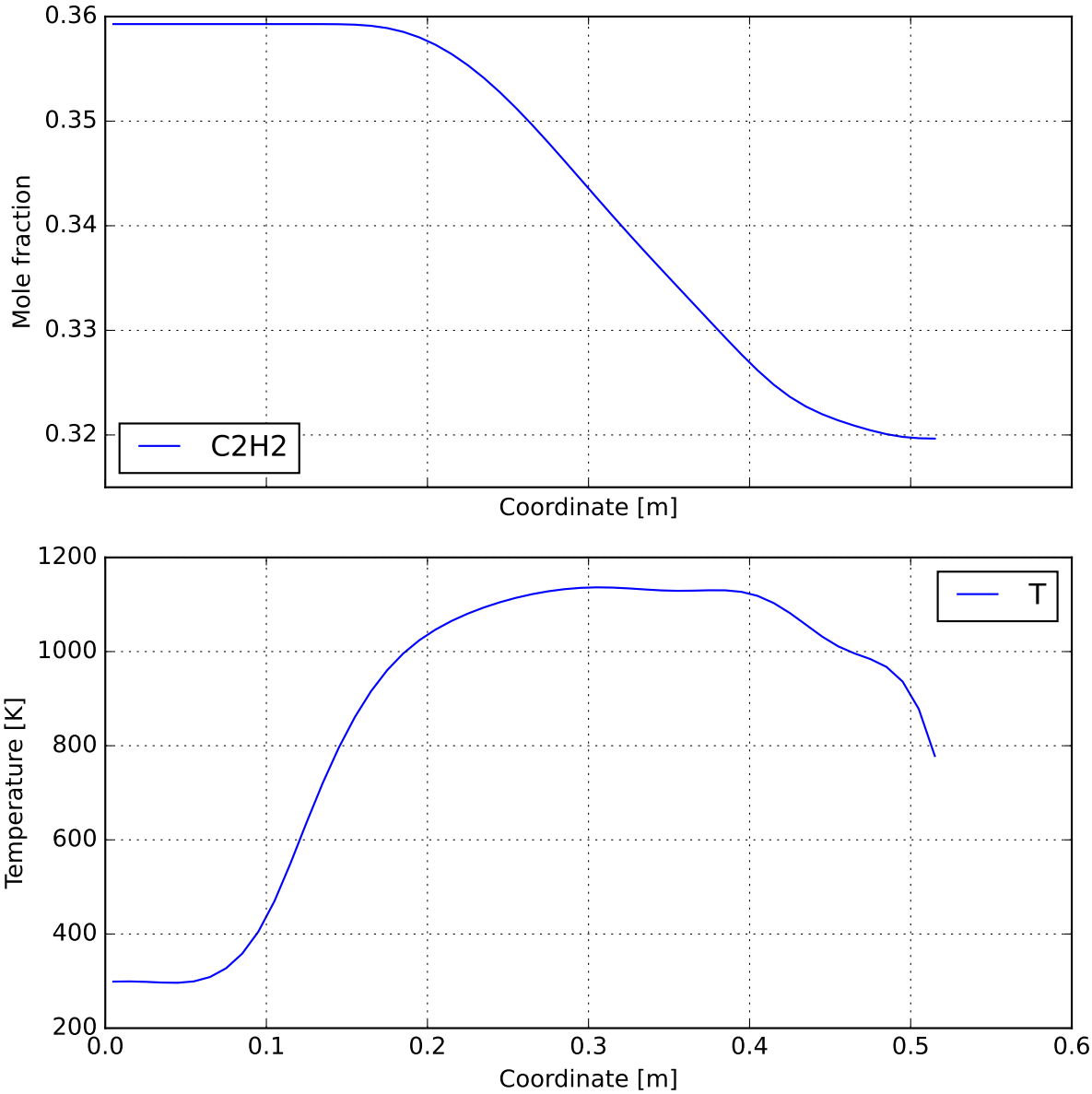
7 Acetylene pyrolysis

> ⚠ Warning
>
> Work in progress!

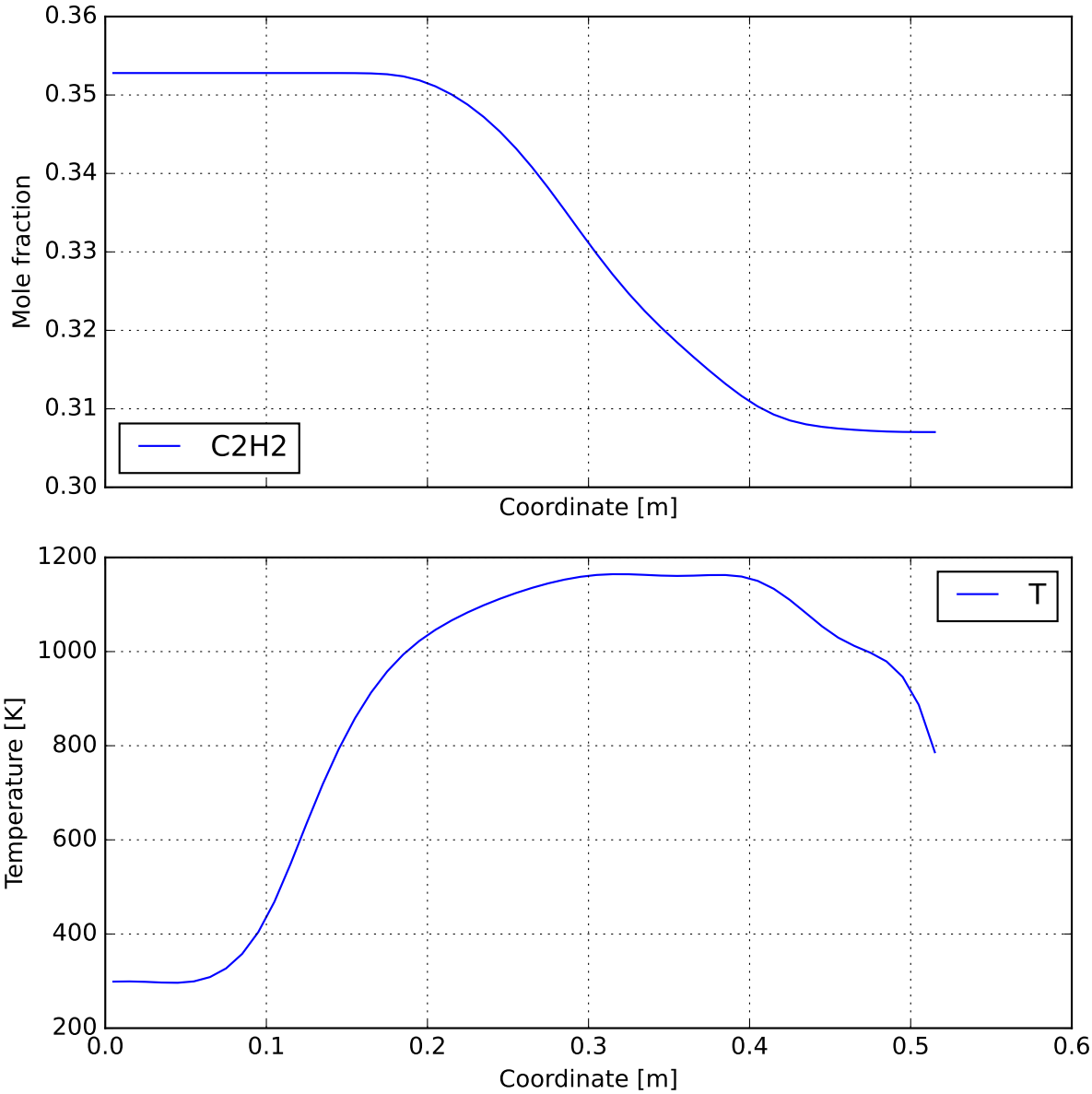
7.1 Toolbox

7.2 Reference case

```
CPU times: total: 750 ms
Wall time: 759 ms
```




```
CPU times: total: 1.86 s
Wall time: 1.87 s
```



7.3 Scan table

```
Working on case no. 1
Working on case no. 2
Working on case no. 3
Working on case no. 4
Working on case no. 5
Working on case no. 6
Working on case no. 7
Working on case no. 8
Working on case no. 9
Working on case no. 10
Working on case no. 11
Working on case no. 12
Working on case no. 13
```

```
Working on case no. 14
CPU times: total: 25.5 s
Wall time: 25.5 s

Working on case no. 1
Working on case no. 2
Working on case no. 3
Working on case no. 4
Working on case no. 5
Working on case no. 6
Working on case no. 7
Working on case no. 8
Working on case no. 9
Working on case no. 10
Working on case no. 11
Working on case no. 12
Working on case no. 13
Working on case no. 14
CPU times: total: 10.4 s
Wall time: 10.3 s
```

	N	P	Q	T	X_meas	X_dalmazsi_2017	err_dalmazsi_2017	err_graf_2007
0	1	50	222	773	0.352	0.352786	0.223212	2.032933
1	2	50	222	873	0.364	0.352540	-3.148330	-1.550413
2	3	50	222	973	0.364	0.350258	-3.775208	-2.602299
3	4	50	222	1073	0.346	0.339812	-1.788298	-1.188026
4	5	50	222	1123	0.312	0.326556	4.665512	6.179187
5	6	50	222	1173	0.307	0.307034	0.010994	4.120820
6	7	50	222	1273	0.288	0.290866	0.995060	6.743916
7	8	30	222	1173	0.323	0.329143	1.901911	5.859843
8	9	30	222	1223	0.314	0.318436	1.412680	7.828360
9	10	100	222	1173	0.249	0.242491	-2.613868	4.125649
10	11	100	222	1223	0.226	0.230372	1.934472	5.944962
11	12	100	222	1273	0.201	0.221251	10.075293	12.714431
12	13	100	378	1023	0.343	0.342777	-0.065090	0.254930
13	14	100	378	1123	0.298	0.305793	2.615003	6.255879

	err_dalmazsi_2017	err_graf_2007
0	3.53261	5.758262

8 Counter-current PFR

> ⚠ Warning
>
> Work in progress!

8.1 Introduction

The goal of this tutorial is to discuss a sequence of increasingly complex plug-flow reactor (PFR) models. These are all built upon `PlugFlowChainCantera` which approximates a PFR as a cascade of perfect-stirred reactors (PSR), as illustrated here. Care must be taken when using such an approach because this zero-dimensional modeling does not formally represent a PFR, but approaches one - so grid convergence studies might be required to validate a model, what is beyond our scope here (and has been done during the conception of this report), which limits itself to studying the coupling of reactors. Nonetheless, this representation proves very interesting when assembling complex systems as it remains modular, allowing for extensibility of models.

In what follows we will be guided through the following models:

- A single PFR with possibility of distributed mass sources along its length; this case is often encountered in industry when modeling reactors coupled through a permeable membrane, or in a rotary kiln, where solids may eventually release gases, what is especially important in the field of biomatter gaseification.
- Next we will couple a pair of reactors as conceived in the first example, which will be allowed to exchange energy through a pseudo-convective model; there is no major difficulty at this point as the coupling can be stabilized through a relaxation routine. We investigate how the ordering of reactor solution affects convergence rate.
- At the last level of complexity studied here, we add a shared common wall around the pair of reactors of the previous step; one might argue that such a system would require two *wall* temperatures to be described at steady state; here we will assume that this wall is *rotating* around the pair of reactors in such a way that it remains in contact with a reactor during part of a cycle, and a *statistically significant mean temperature* is solely used as a process measurable.

Because writting long classes is not didatic for the purposes of a tutorial, we will use a *semi-object-oriented* programming approach here. That means, there will be minimal classes just holding the basic elements representing the state of a problem, and functions that resemble class methods (receiving the class instance as first argument) to perform the required operations.

Below we start by importing all required tools at once:

Because some cells take some time to execute, during development you can set this to `"true"` so that they are not run. You can recognize these cells by the presence of magic `%%skipper`. Setting this to `"false"` will not skip the cells (this is the logic of the booleans here, we are saying true to skip, not true to execute!).

8.2 Shared functionalities

This section is not commented, variables and functions should speak by themselves.

8.3 Single plug-flow reactor

For the single reactor `ReactorModel` we write a simple wrapper around `PlugFlowChainCantera`; this model holds an instance of a `solution` object conceived with the same mechanism as provided to the PFR, and a simple `source` structure for the distributed source terms.

Both the `solution` and `source` are used in `add_source` to inject a mass flow rate `mdot` at temperature `T`, and composition `X` (given in mole fractions) at a set of cell indices `where` along the reactor. We will see later how discretization is performed and thus how to link `where` to an actual axial coordinate along the reactor.

The underlining PFR model has already its own solution method (which is actually called `update`) that *seeks* a steady state for the reactor given a certaing `source`; sometimes it fail to converge and multiple calls may be required to find the steady state of a reactor. In `solve_reactor` below we wrap the aforementioned method and add some reporting.

8.3.1 Example - single PFR simulation

As this first case is already fully supported by the underlining library class, we are ready to solve a first example problem. Comments are provided in the function `sample_single` to guide you through the setup. A distributed power supply is added to the reactor as an analytical function as:

$$Q(z) = \begin{cases} A\left[1 - \exp\left(-\frac{z}{z_0}\right)\right] & z < \frac{L}{2} \\ 0 & \text{otherwise} \end{cases}$$

This is arbitrarily parametrized and is intended just to show that accessing `model.source.Q` allows to provide any heat source function to a reactor.

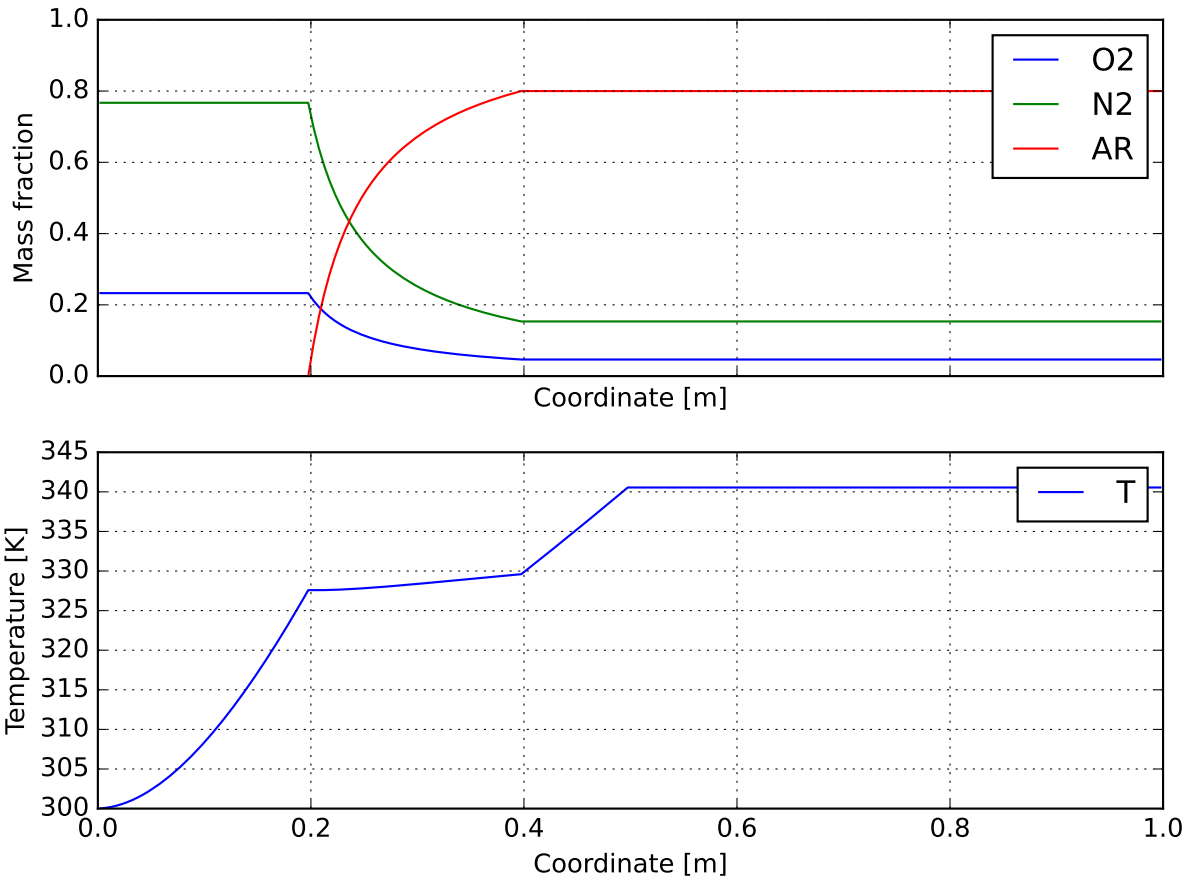
We said no convergence study was going to be performed; that was a lie. Below we inspect the evolution of final temperature in terms of reactor cell length in range $l \in [0.001; 0.1]$ m. Solution quickly approaches 340.6 K, and for practical purposes a discretization of 1 cm seems a good compromise between execution time and accuracy. Also notice that total supplied power may depend on discretization because of how it is provided.

```
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +2.500000e+02 |
| Total external power | W   | +6.035993e+03 |
| Final temperature    | K   | +3.404046e+02 |

|----------------------|-----|---------------|
| Total mass flow rate | g/s | +2.500000e+02 |
| Total external power | W   | +6.000357e+03 |
| Final temperature    | K   | +3.405258e+02 |
```

Below we show the report of first solution, which here is no better than an initial guess; sequential solution of reactor towards steady state is recommended in practice.

```
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +2.500000e+02 |
| Total external power | W   | +6.000085e+03 |
| Final temperature    | K   | +3.395247e+02 |
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +2.500000e+02 |
| Total external power | W   | +6.000085e+03 |
| Final temperature    | K   | +3.405501e+02 |
```

Finally we illustate the composition and temperature profiles induced by the simultaneous mass and heat transfer:



You can also access the data in base reactor for post-processing as desired:

	0	1	2	3	4
z_cell	0.002500	0.007500	0.012500	0.017500	0.022500
V_cell	0.000039	0.000039	0.000039	0.000039	0.000039
Q_cell	1.177767	3.489592	5.744338	7.943414	10.088195
m_cell	0.000046	0.000046	0.000046	0.000046	0.000046
mdot_cell	0.050000	0.050000	0.050000	0.050000	0.050000
T	300.023321	300.092416	300.206156	300.363436	300.563179
density	1.171880	1.171610	1.171166	1.170553	1.169775

	0	1	2	3	4
Y_O2	0.232909	0.232909	0.232909	0.232909	0.232909
Y_N2	0.767091	0.767091	0.767091	0.767091	0.767091
Y_AR	0.000000	0.000000	0.000000	0.000000	0.000000

8.4 Counter-current flow reactors

The tooling required to compute a pair of reactors is already provided by `ReactorModel`; all we have to do is wrap a pair of reactors in `CounterCurrentReactors`. For convenience we store a copy of the axial coordinate (which is the same for both reactors). Notice here the introduction of `U`, a global heat transfer coefficient between phases, `Q`, the associated heat flux, and a relaxation manager.

One cannot compute heat fluxes between reactors before having an initial temperature profile established (see it as an initial guess). Function `initialize` is provided for that end and must be called after mass flows have been setup.

There are two ways one can solve the relaxation problem here, each with its strengths and weaknesses; one could simply calculate calculate the flux, apply it to both reactors, and solve (`iter_direct`) or compose a series of flux and solution calculations (`iter_alternate`). When initial guess is not good enough, the alternating approach tends to deliver results in a more reliable fashion (because it is compatible with high relaxation factors), but it is slower. On the other hand, direct solution may converge faster as solution is already relaxed.

IMPORTANT: notice here that we get the reverse of `T2` to compute flux (as the reactors are in counter-current) and then `model.Q` is reversed againt to set heat flux to reactor 2. This class could easily be generalized by adding a flag on whether temperature and flux should be treated as is (co-current) or reversed.

Based on the previous discussion, a mixed approach can be proposed: relax the problem with the alternating solution and move to direct one after a certain criteria or maximum number of iterations has been achieved. This is emulated in `solve_pair` provided below.

A simple standard ploting function is provided for the counter-current reactor pair:

8.4.1 Example - pair of coupled PFR

This second example replaces the user-defined heat flow function by a convective heat exchange through a fin separating the reactions; this is modeled through a heat transfer coefficient `h` which is made constant here for illustration purposes only. To test the robustness of the approach, a relativelly high dilution flow is applied to a concentrated region in reactor 2 (hot stream).

After building the reactor pair, inlet and distributed conditions and the reactors are initialized, *.i.e.* solved individually, before iterative solution is performed. The user may require a final solution step for printing reactor quantities in a table.

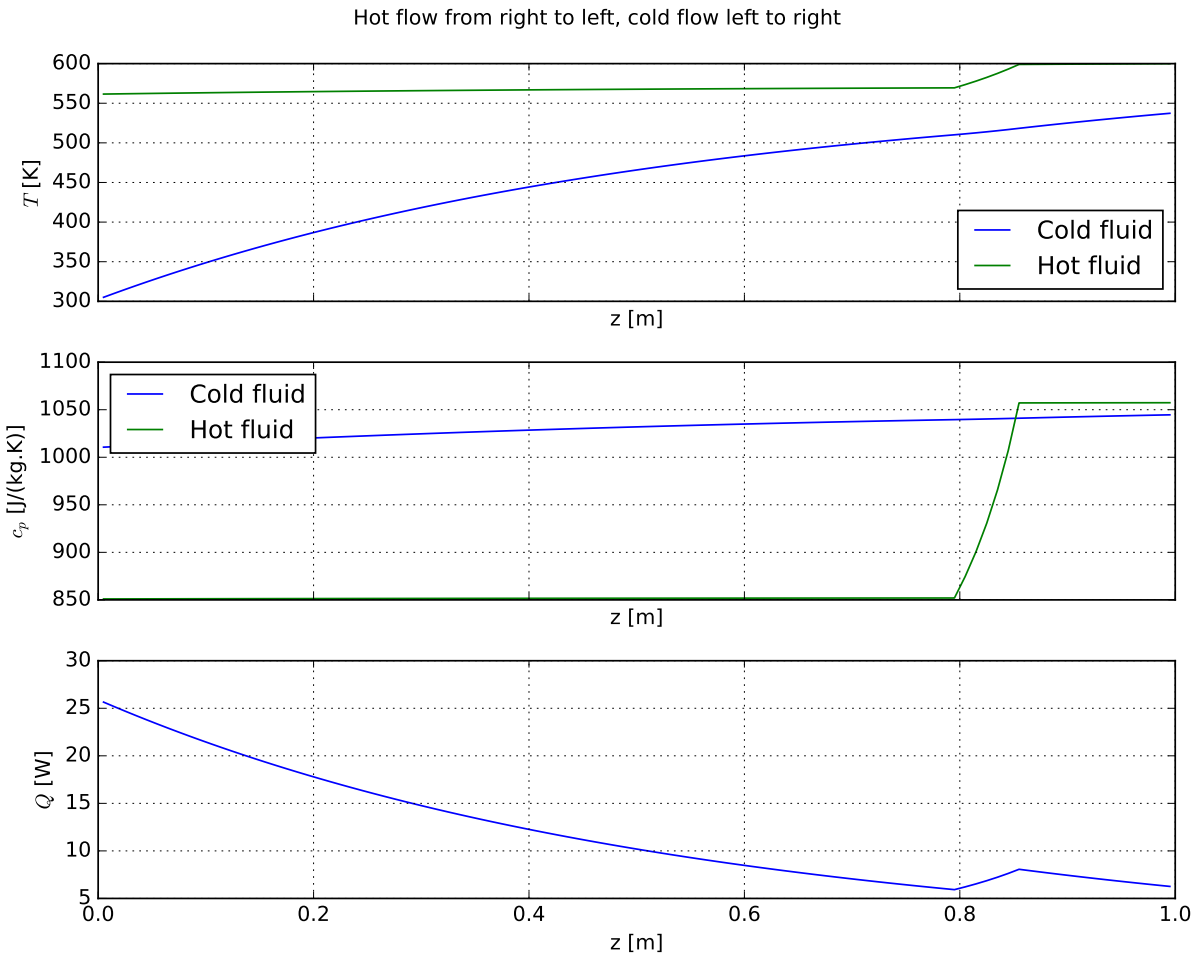
For low relaxation values ($\alpha < 0.5$), both methods may eventually converge under the same number of iterations.

```
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +5.000000e+00 |
| Total external power | W   | +1.216871e+03 |
```

```
| Final temperature     | K   | +5.372994e+02 |
|-----------------------|-----|---------------|
| Total mass flow rate | g/s | +1.600000e+02 |
| Total external power | W   | -1.216871e+03 |
| Final temperature     | K   | +5.615588e+02 |
```

Refining the grid for the present case has an impact of a few degrees over final solution (but takes much longer to solve), as illustrated below. Below 2 mm there is little change in the final solution and this value is a practical limit for grid refinement here.

Finally we display the results; for the hot stream we observe a sharp decrease in temperature (and associated specific heat capacity) due to argon dilution. Cold stream progressivelly heats up along the channel length as expected.



8.5 Adding losses to the environment

So far we have developed the ideas required to reach the final goal of conceiving a fully coupled system with a pair of reactors and walls. Again we proceed with a wrapper class which countains our problem variables, but in this case we fully develop it as a single block. Also, this time documentation is provided in the class itself as that seems clearer to the author in this case, and provides some closure to the study.

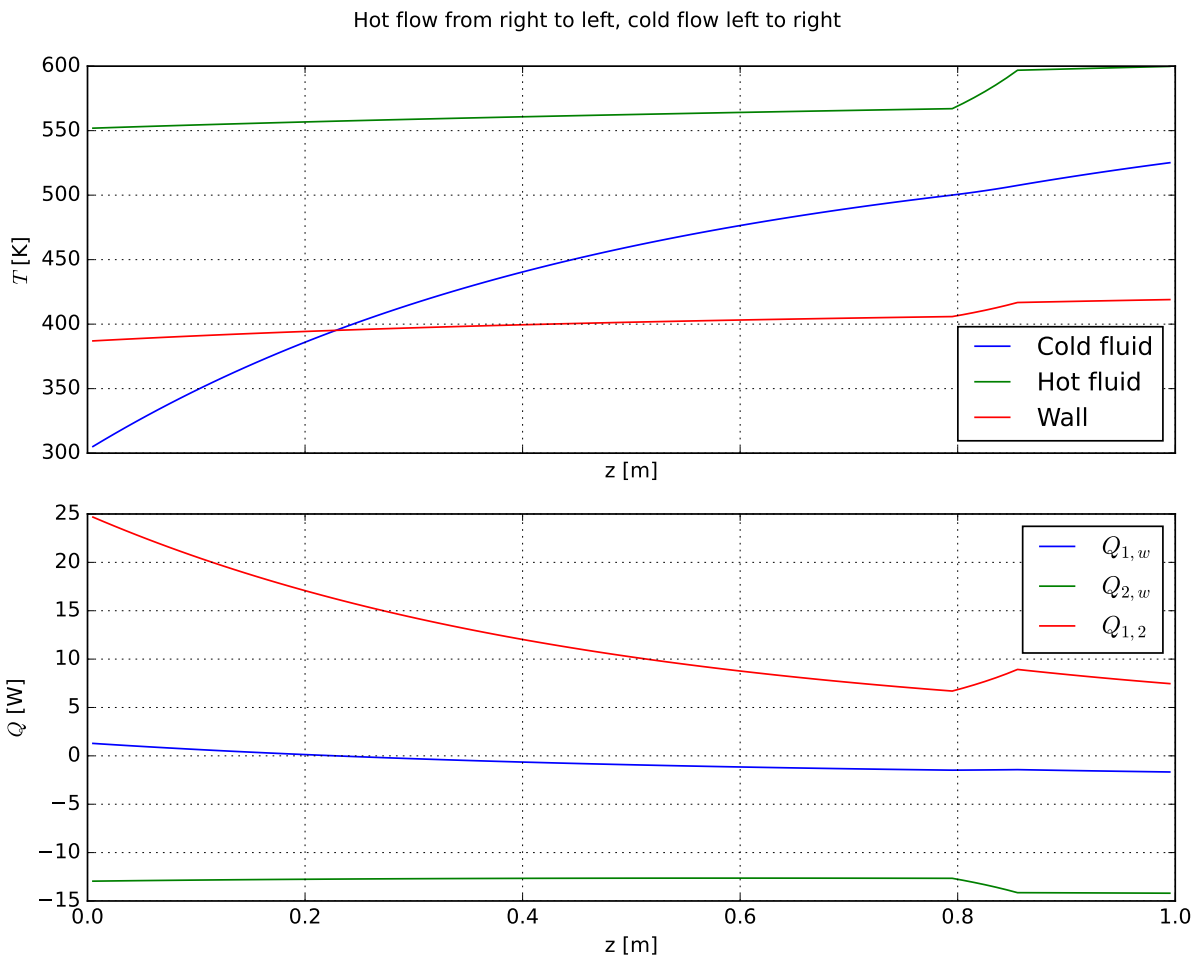
The main increment in problem solution here is the extra step of solving for the wall temperature, which can be found from the global flux definition:

$$\dot{Q}_w = \frac{T_w - T_e}{R_{tot}} \quad \text{where} \quad R_{tot} = \frac{1}{hA} + \frac{1}{2\pi lk} \log \frac{R_{ext}}{R_{int}}$$

We make use of the main takes of the previous sections, which have shown that alternating reactor solution order is not always an advantage; here the problem has been programmed to use direct solution. Relaxation is applied to reactor fluxes, but not to wall flux, which is computed from a balance of the relaxed quantities for the sake of energy conservation.

8.5.1 Example - environment coupling

```
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +5.000000e+00 |
| Total external power | W   | +1.003972e+03 |
| Final temperature    | K   | +4.963894e+02 |
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +3.000000e+02 |
| Total external power | W   | -2.210504e+03 |
| Final temperature    | K   | +5.216989e+02 |

|----------------------|-----|---------------|
| Total mass flow rate | g/s | +5.000000e+00 |
| Total external power | W   | +1.153462e+03 |
| Final temperature    | K   | +5.251459e+02 |
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +1.600000e+02 |
| Total external power | W   | -2.516963e+03 |
| Final temperature    | K   | +5.519201e+02 |
```

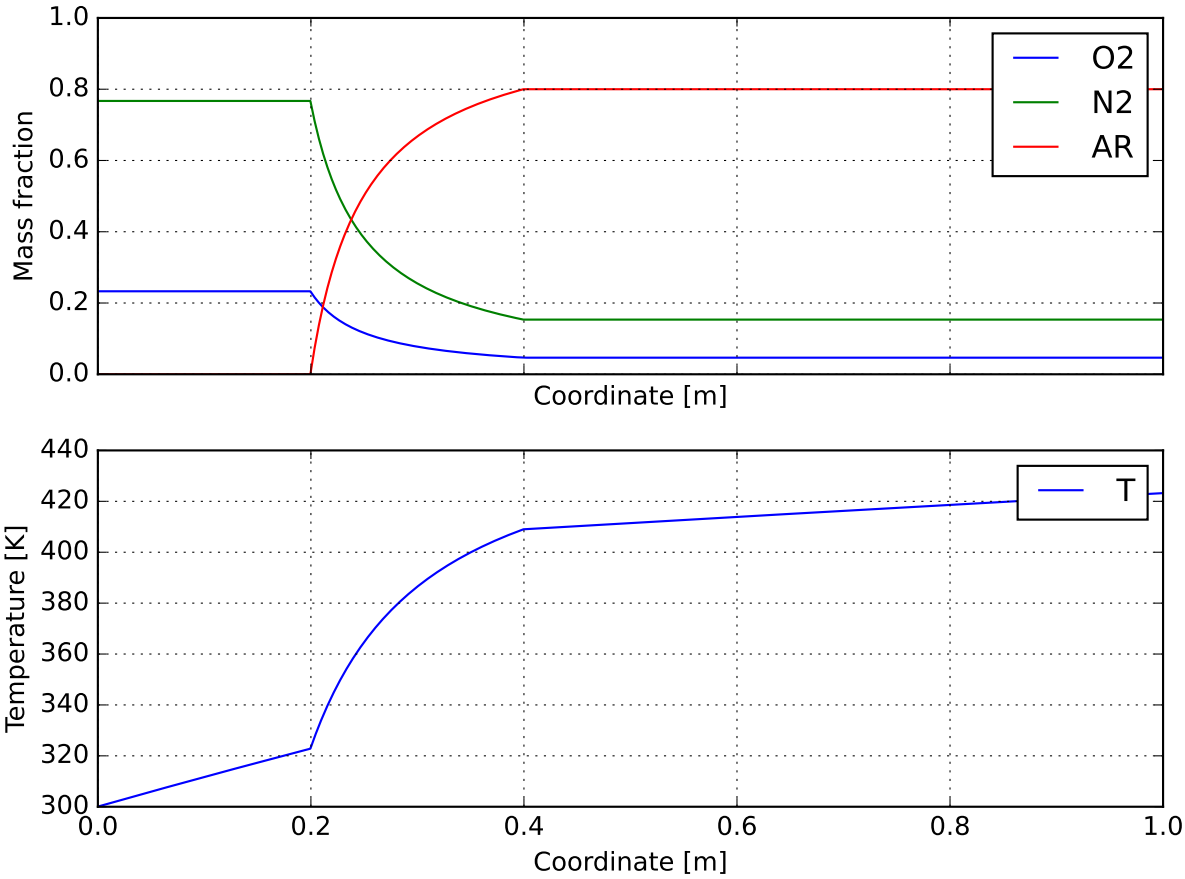


8.6 Registering flow for a single reactor

Below we develop further the idea into registering heat flows as an alternative to relaxation. The method may prove useful because heat flows are computed from current state of the reactors instead of an external guess.

8.6.1 Example - prototype with a single reactor

```
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +2.500000e+02 |
| Total external power | W   | +4.237303e+03 |
| Final temperature    | K   | +4.232112e+02 |
```



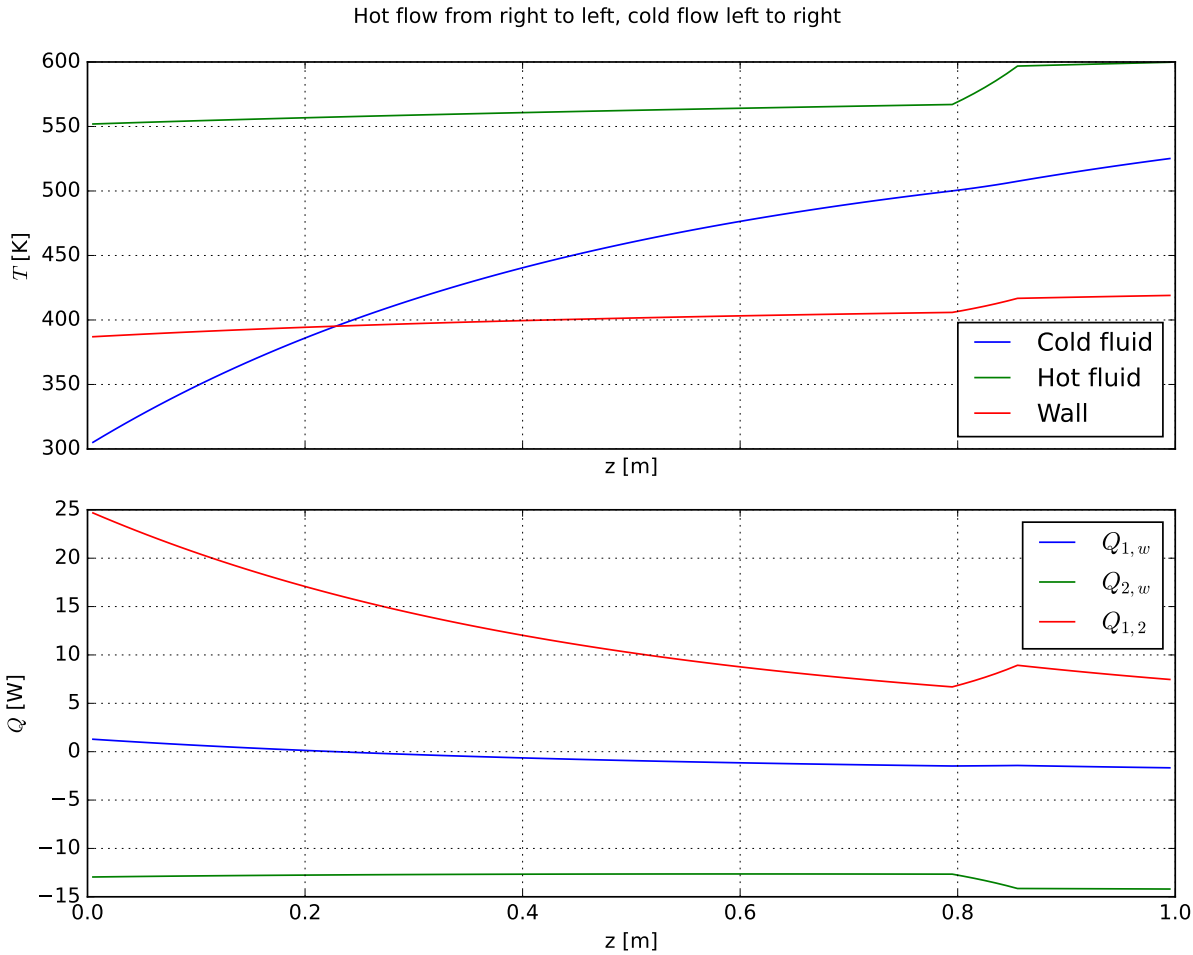
8.7 Registering flow for a pair of reactors

As a last level of improvement, we generalize the approach for a pair of reactors. After some experimentation it was concluded that falling back to the alternating approach with intermediate wall solution lead to the best performance, probably because of decreased system stifness. Relaxing the wall heat flow allows for faster convergence.

8.7.1 Example - extension to coupled system

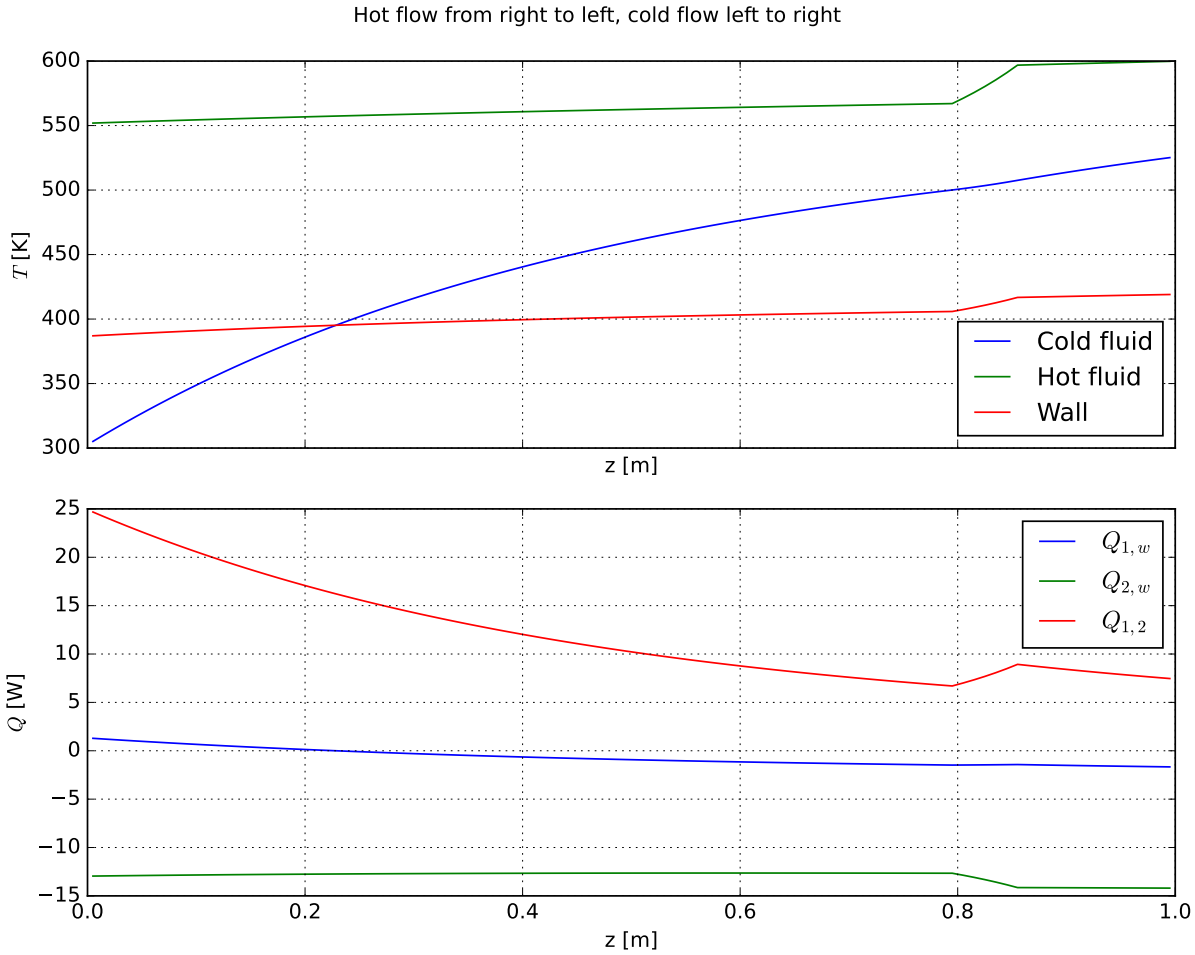
```
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +5.000000e+00 |
| Total external power | W   | +1.153462e+03 |
| Final temperature    | K   | +5.251459e+02 |
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +1.600000e+02 |
| Total external power | W   | -2.516963e+03 |
| Final temperature    | K   | +5.519201e+02 |
CPU times: total: 15.5 s
Wall time: 15.6 s

|----------------------|-----|---------------|
| Total mass flow rate | g/s | +5.000000e+00 |
| Total external power | W   | +1.153462e+03 |
```

```
| Final temperature     | K    | +5.251459e+02 |
|-----------------------|------|---------------|
| Total mass flow rate | g/s  | +1.600000e+02 |
| Total external power | W    | -2.516963e+03 |
| Final temperature     | K    | +5.519201e+02 |
CPU times: total: 17 s
Wall time: 17.1 s
```

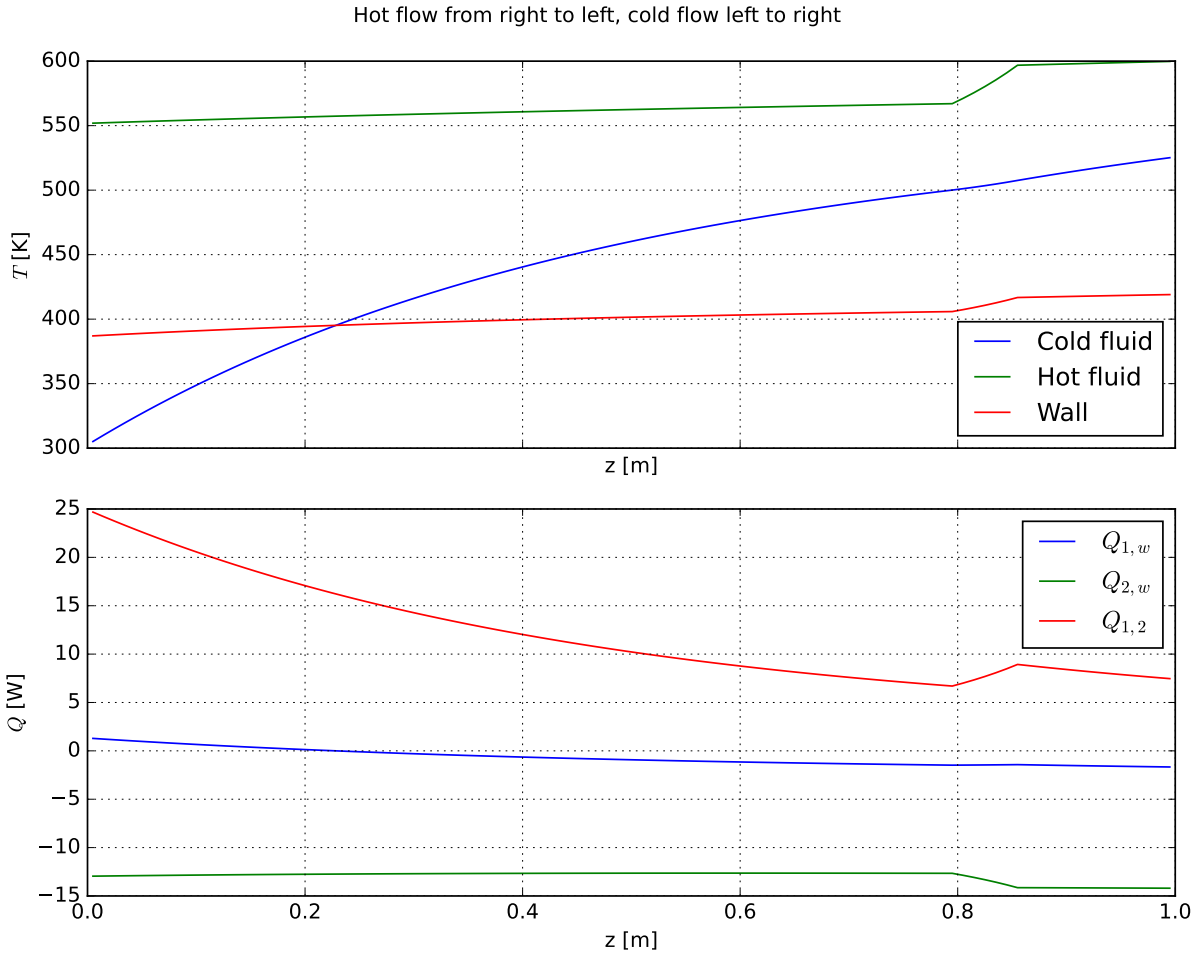



```
|-----------------------|------|---------------|
| Total mass flow rate | g/s  | +5.000000e+00 |
| Total external power | W    | +1.153462e+03 |
| Final temperature     | K    | +5.251459e+02 |
|-----------------------|------|---------------|
| Total mass flow rate | g/s  | +1.600000e+02 |
| Total external power | W    | -2.516963e+03 |
| Final temperature     | K    | +5.519201e+02 |
CPU times: total: 15 s
Wall time: 15 s

|-----------------------|------|---------------|
| Total mass flow rate | g/s  | +5.000000e+00 |
| Total external power | W    | +1.153462e+03 |
| Final temperature     | K    | +5.251459e+02 |
```

```
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +1.600000e+02 |
| Total external power | W   | -2.516963e+03 |
| Final temperature    | K   | +5.519201e+02 |
CPU times: total: 15.9 s
Wall time: 15.9 s
```




```
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +5.000000e+00 |
| Total external power | W   | +1.153462e+03 |
| Final temperature    | K   | +5.251459e+02 |
|----------------------|-----|---------------|
| Total mass flow rate | g/s | +1.600000e+02 |
| Total external power | W   | -2.516963e+03 |
| Final temperature    | K   | +5.519201e+02 |
CPU times: total: 3.55 s
Wall time: 3.53 s
```


Hot flow from right to left, cold flow left to right
600
550
500
450
400
350
300
T [K]
z [m]
Cold fluid
Hot fluid
Wall
25
20
15
10
5
0
−5
−10
−15
Q [W]
0.0
0.2
0.4
0.6
0.8
1.0
z [m]
$Q_{1,w}$
$Q_{2,w}$
$Q_{1,2}$

9 Fitting NASA9 data

> ⚠ Warning
>
> Work in progress!

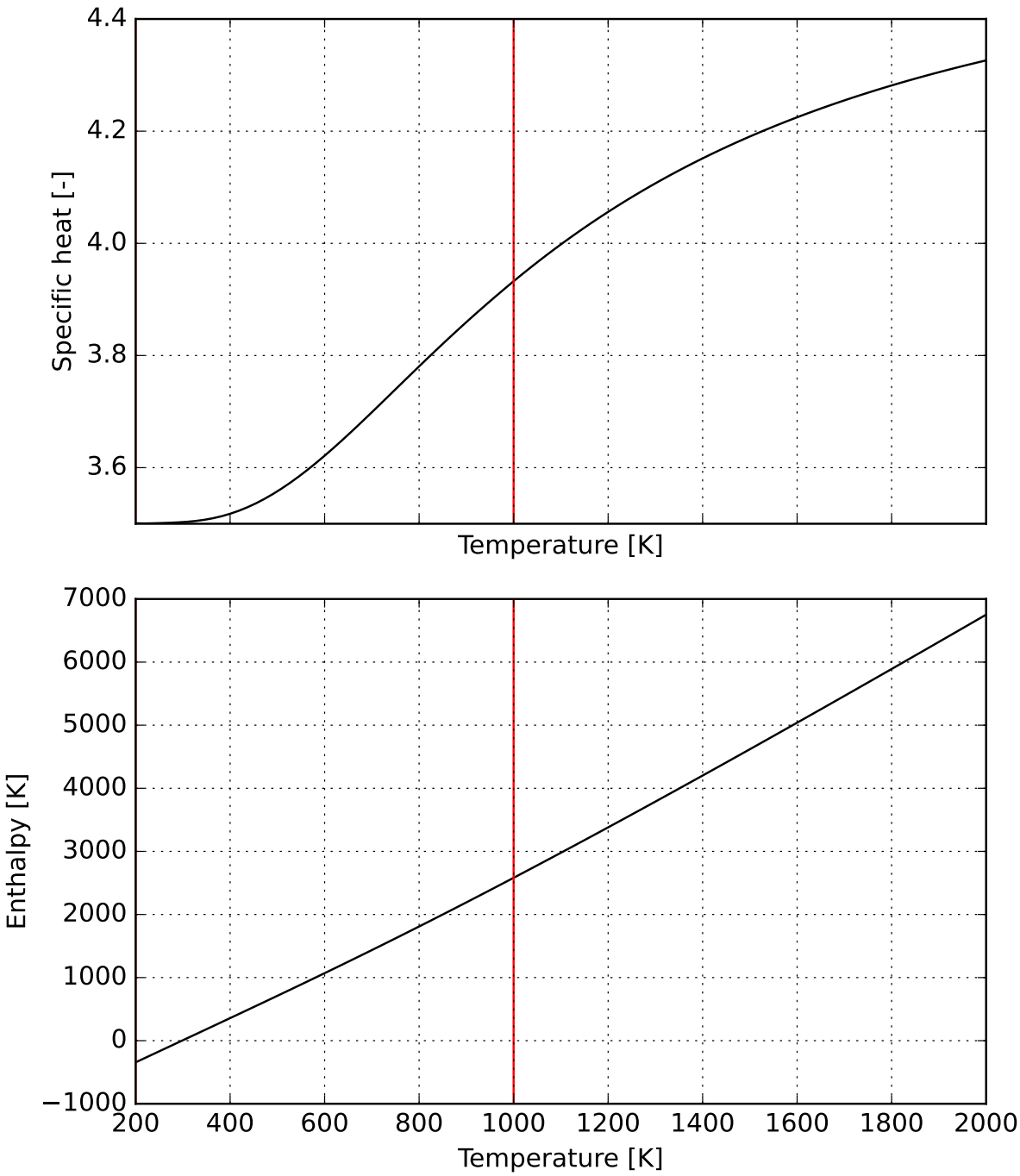
The goal of this document is to provide a better understanding on how to fit empirical data to NASA9 parameterization of thermodynamic properties; more details are provided in the original NASA report. A progressive level of complexity is used to provide better insights of how the species thermodynamics is encoded in this format.

9.1 Helper utilities

9.2 Nitrogen actual fit

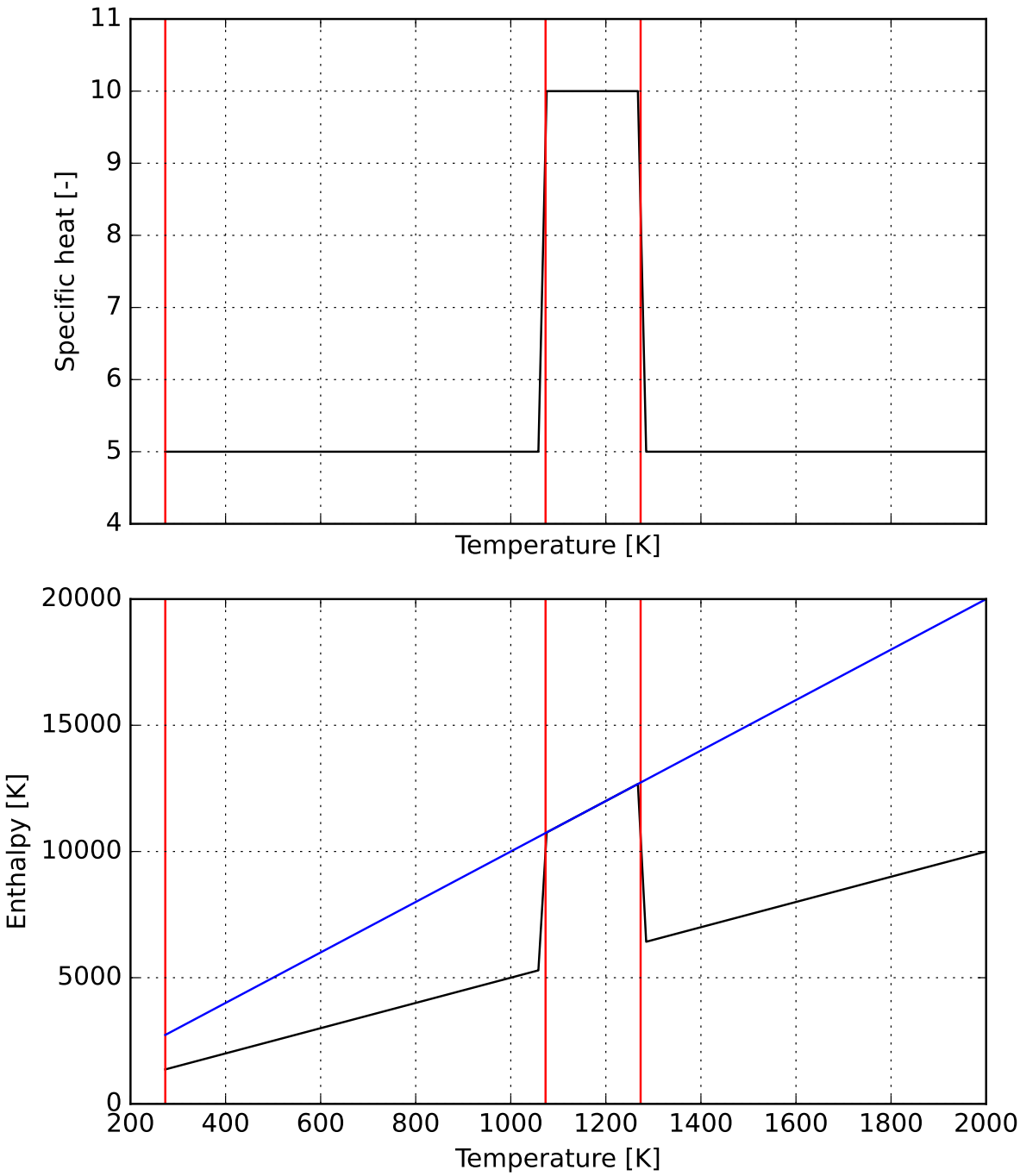
- Below we display in text format an actual NASA9 fit taken from `airNASA9.yaml` data file provided with Cantera; this is a fully parameterized species, making it a bit more difficult to visually understand the role of different coefficients
- The third coefficient a_2 is related to the specific heat intercept when $T = 0$ and the eighth coefficient a_7 represents the integration constant for providing enthalpy over a range
- Since NASA9 functions are provided in dimensionless form, it is worth dividing computed values by ideal gas constant, so that values are in the same *units* as the provided coefficients

The next cell illustrates the dimensionless specific heat and enthalpy over a typical industrial range.



9.3 Enforcing constant specific heat

- Next we create a dummy species (in Cantera composition needs to include *actual* atoms or you need to create a new database for elements) with all coefficients set to zero except those related to specific heat
- This way we establish a piecewise-constant specific heat function; notice that doing so will produce thermodynamic inconsistent steps in enthalpy



- First, notice that the temperature ranges are *closed-open* ranges, meaning the first point belongs to the range but the last not; we introduce `T_tol` to compute a *last* value of enthalpy over each range as close as possible to the next
- To ensure continuity of enthalpy function, we evaluate the gap `h_hi-h_lo` at each range end and add that to the correction of the previous range (as enthalpy is integrated from the reference temperature)
- Using `dh0 = s.dimless_h(298.15)` to compute the initial value is optional and simply enforces zero enthalpy at the reference state, but in principle it won't affect any computations as only *enthalpy changes* make sense physically

Note: as we are dealing with constant c_P here, we could have computed all the above enthalpy values straight from specific heat integration by hand, but this would be more complicated in general

cases and it was chosen to present the more general (and with least user interaction) approach. See below that the enthalpy at reference temperature fits nicely with the specific heat provided above.

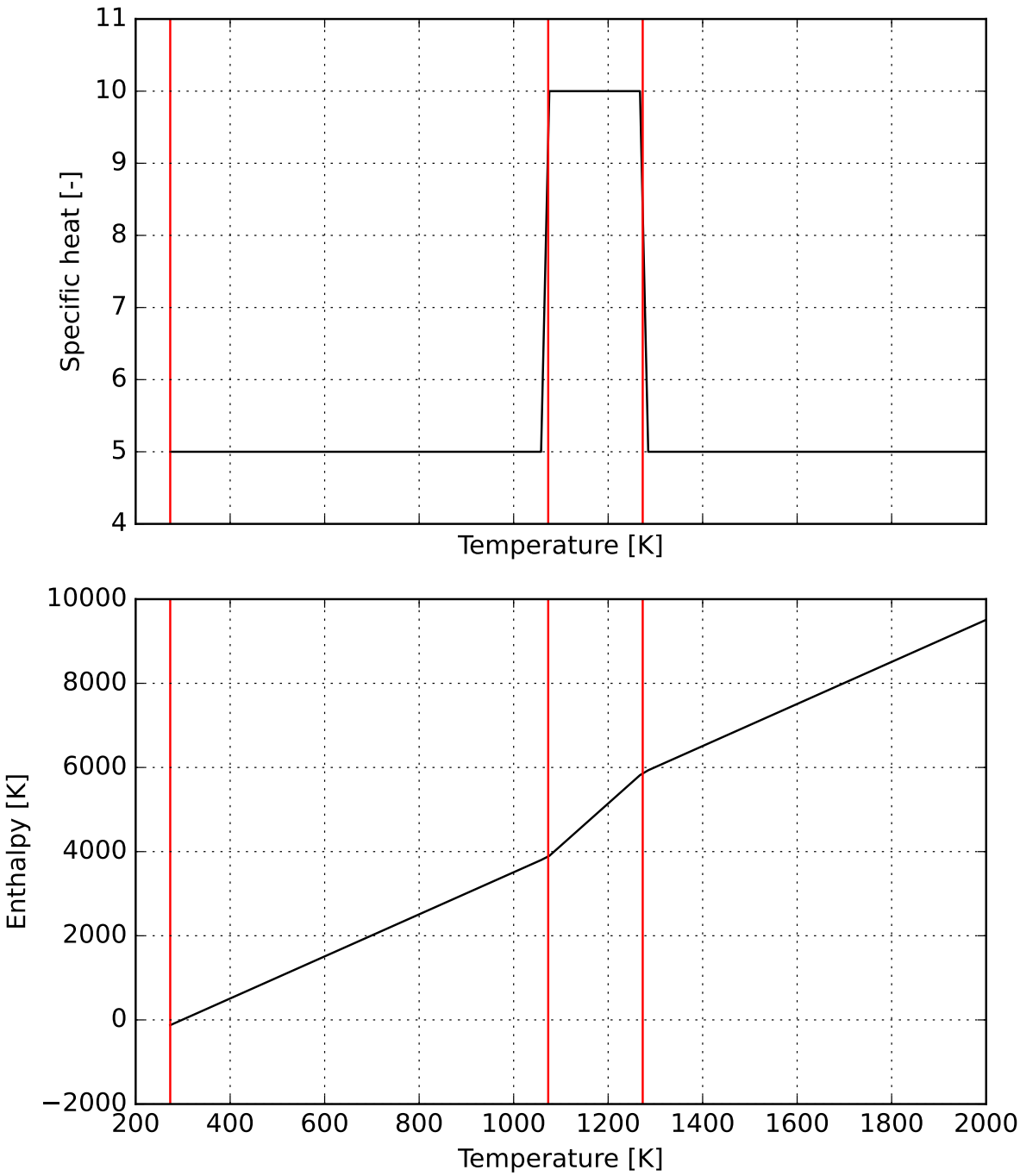
```
np.True_
```

Generalizing for all the steps, we could manually compute everything as follows:

```
((6856.5, np.float64(6856.500000000005)),
 (490.75, np.float64(490.75000000001455)))
```

9.4 Ensuring enthalpy continuity

- To wrap up, we use string interpolation to fix the integration constants required to ensure continuity of enthalpy function



9.5 Application to a real material

We start by providing actual physical values to be converted into NASA9 format.

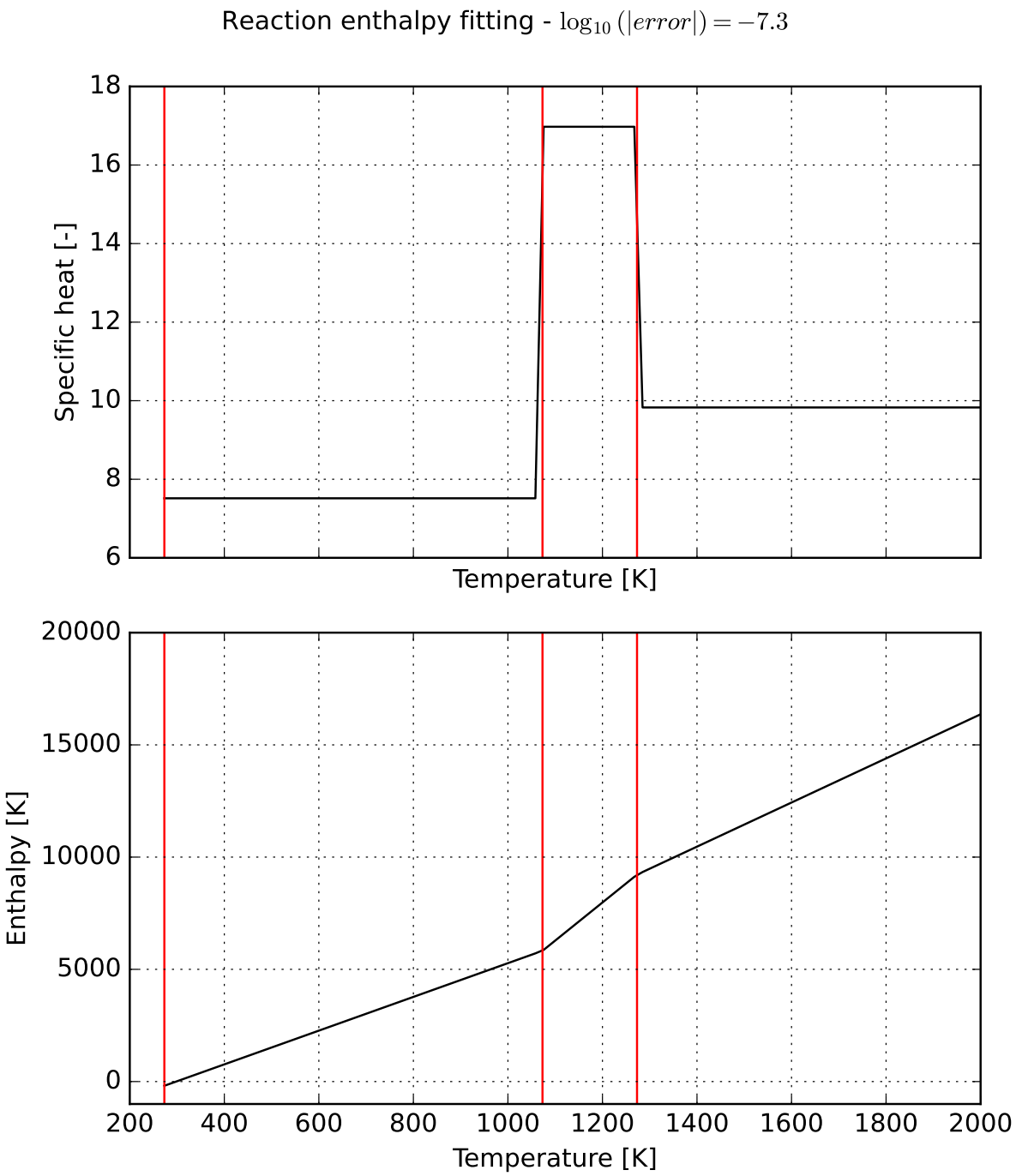
Calculations involve three phases (1) convert into units used by NASA9 format, (2) smearing of reaction enthalpy into specific heat, and (3) corrections required for continuity of enthalpy function.

Alternative for using a constant specific heat over the *whole range*:

Next we generate the YAML input that will be used in Cantera database file.

```
name: meal
composition: {Si: 1, O: 2}
thermo:
  model: NASA9
  temperature-ranges: [273.15, 1073.15, 1273.15, 6000.0]
  data:
  - [ 0.00000000e+00,  0.00000000e+00,  7.51537686e+00,
      0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
      0.00000000e+00, -2.24070961e+03,  0.00000000e+00]

  - [ 0.00000000e+00,  0.00000000e+00,  1.69746349e+01,
      0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
      0.00000000e+00, -1.23919123e+04,  0.00000000e+00]

  - [ 0.00000000e+00,  0.00000000e+00,  9.82780051e+00,
      0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
      0.00000000e+00, -3.29292018e+03,  0.00000000e+00]


name: meal
composition: {Si: 1, O: 2}
thermo:
  model: constant-cp
  cp0: 95.5200 J/mol/K
  h0: 0.0 J/mol®
```

Finally we inspect results before proceeding to use of newly fit data.



Important: NASA9 is the only parameterization supporting an arbitrary number of temperature ranges in Cantera. Given the number of parameters, it is expected to slow down simulations. For more details, please check the documentation.

9.6 Generalizing approach

> ⚠ Warning
>
> *Work in progress from here on... the code is right but the thermodynamics is pretty much sketchy!*

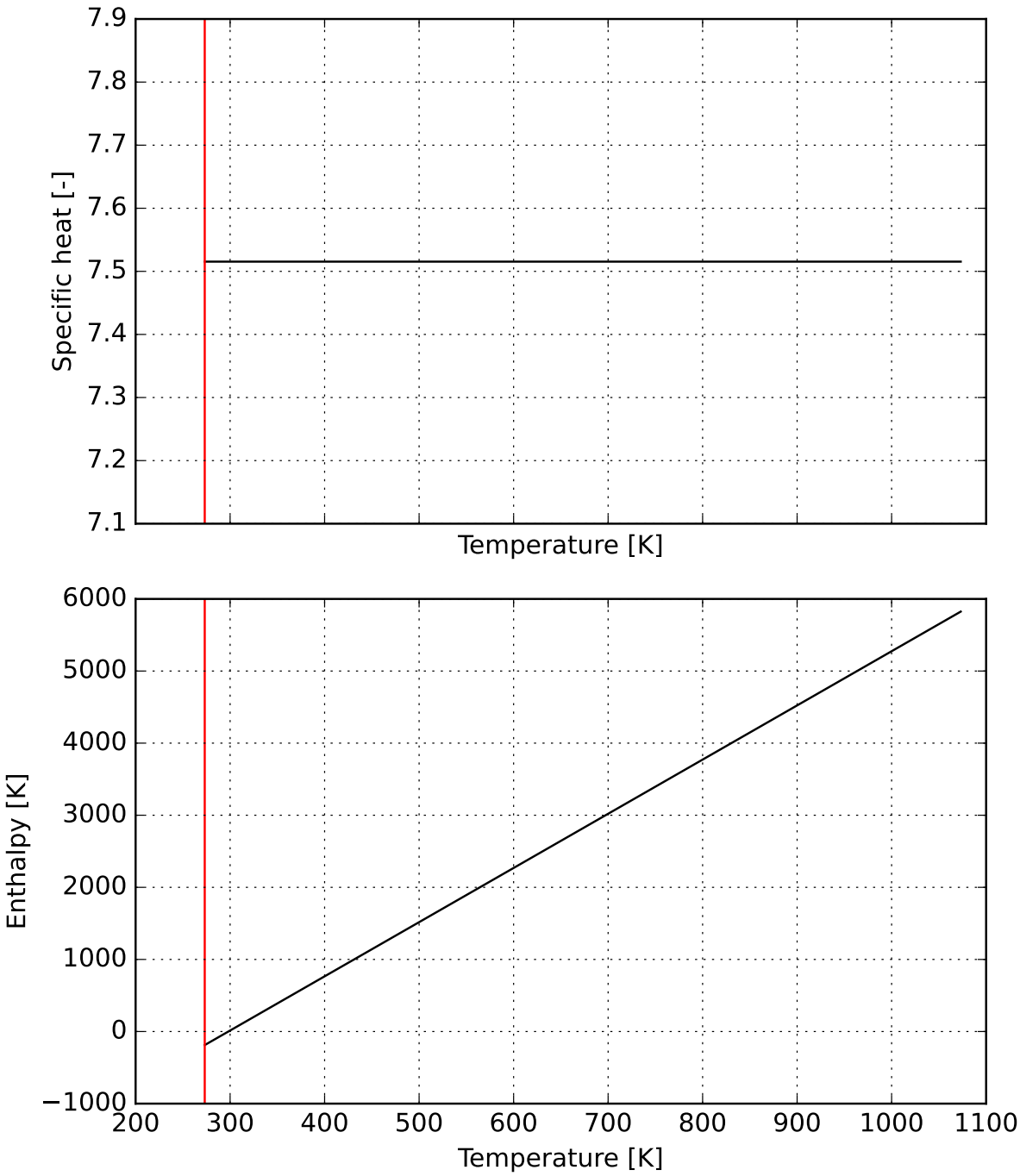
We start by declaring the symbols for temperature `t` and model coefficients `a`:

The next cell provides the representations of specific heat capacity and enthalpy as described here. We also provide the derivatives of theses functions computed algorithmically with help of `casadi.gradient`.

Below we illustrate the symbolic computational graph of operations for checking the functions:

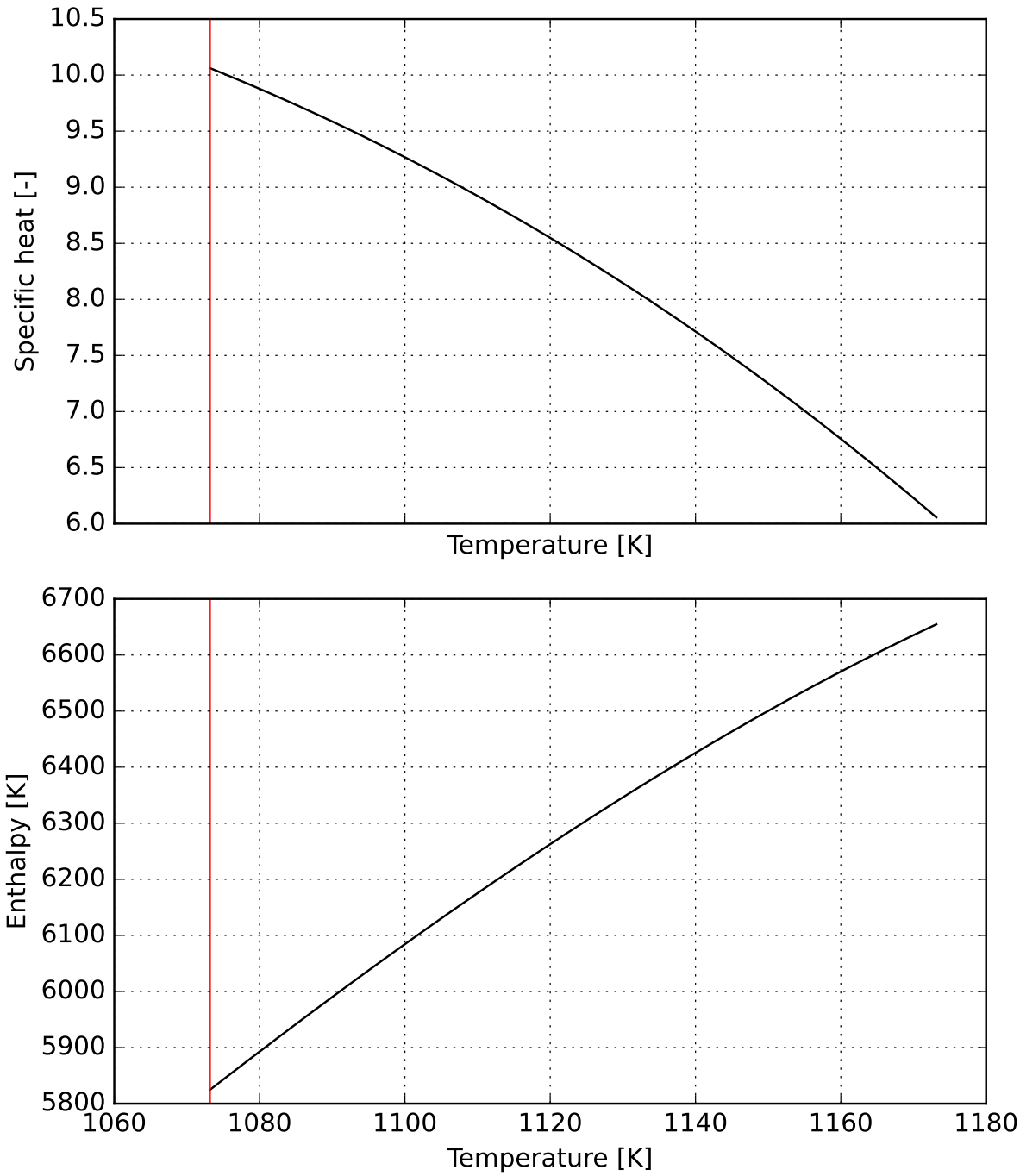
```
SX(((((a_0+(a_1*t))+(a_2*sq(t)))+(a_3*(t*sq(t))))+(a_4*sq(sq(t)))))
```

Below the reaction start we enforce a constant specific heat with zero enthalpy at reference temperature:



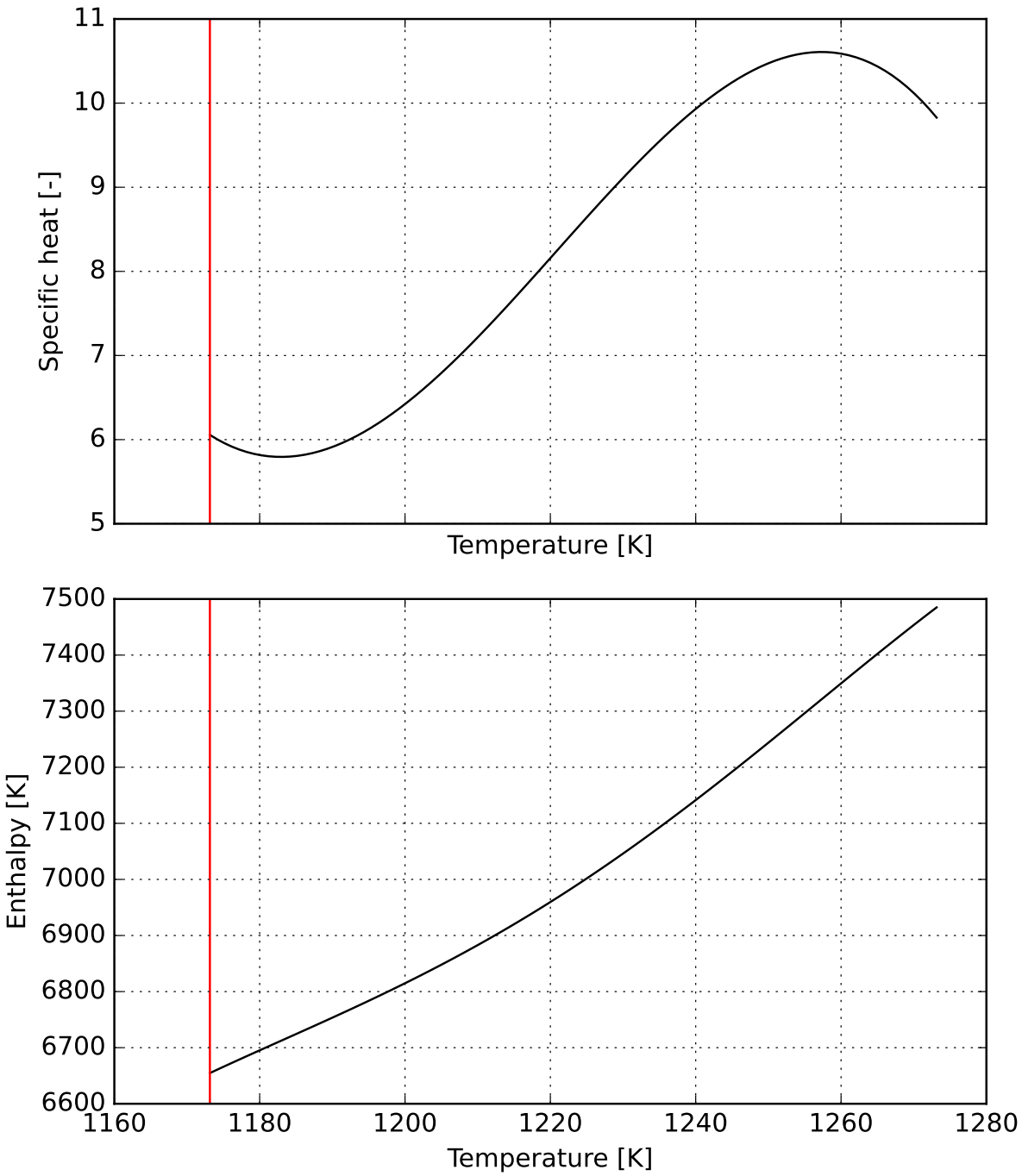
The reaction range is split in two parts, the first corresponding to a raising specific heat which reaches the maximum value.

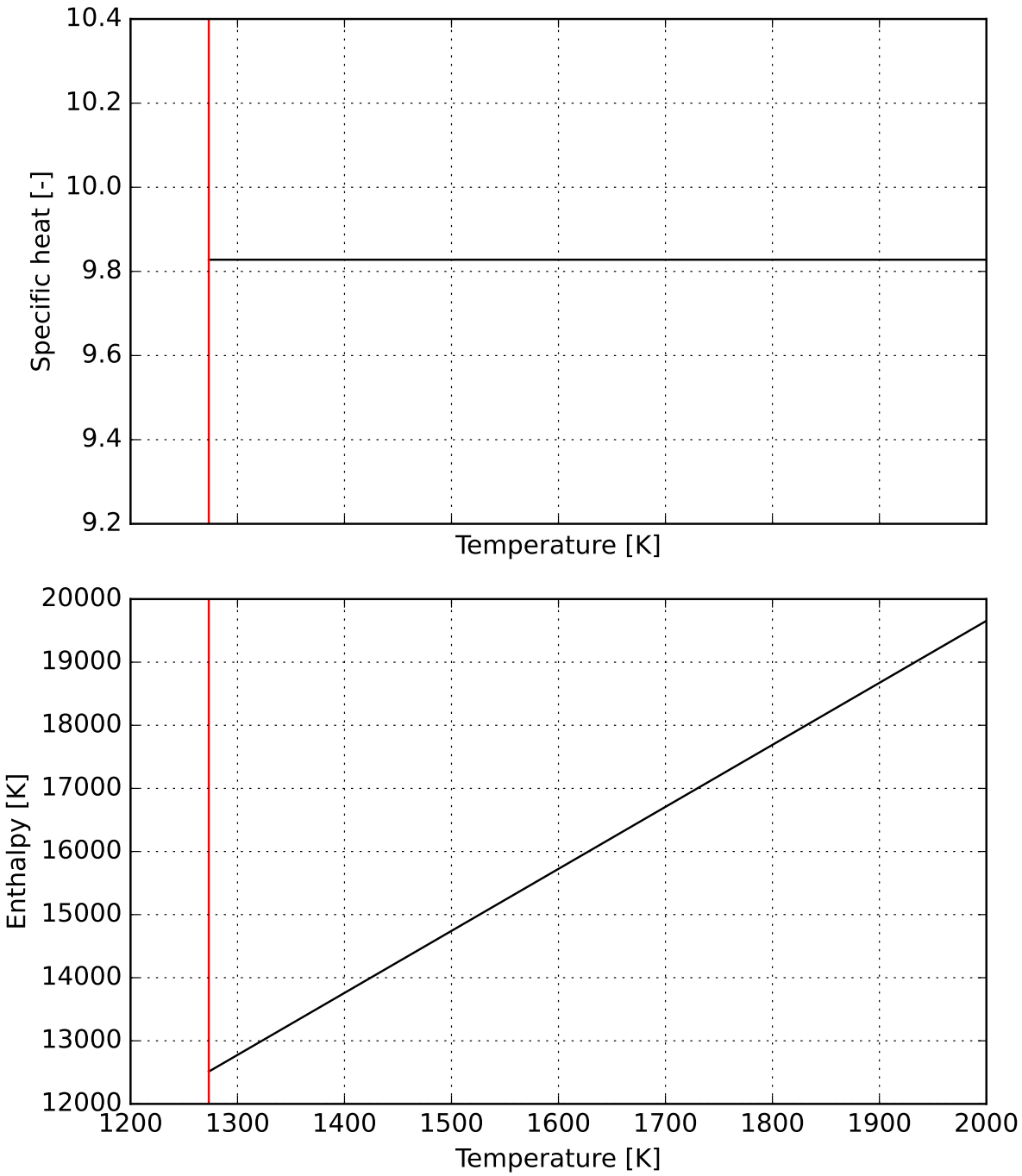
```
DM([1.81899e-12, -3.41061e-13])
```

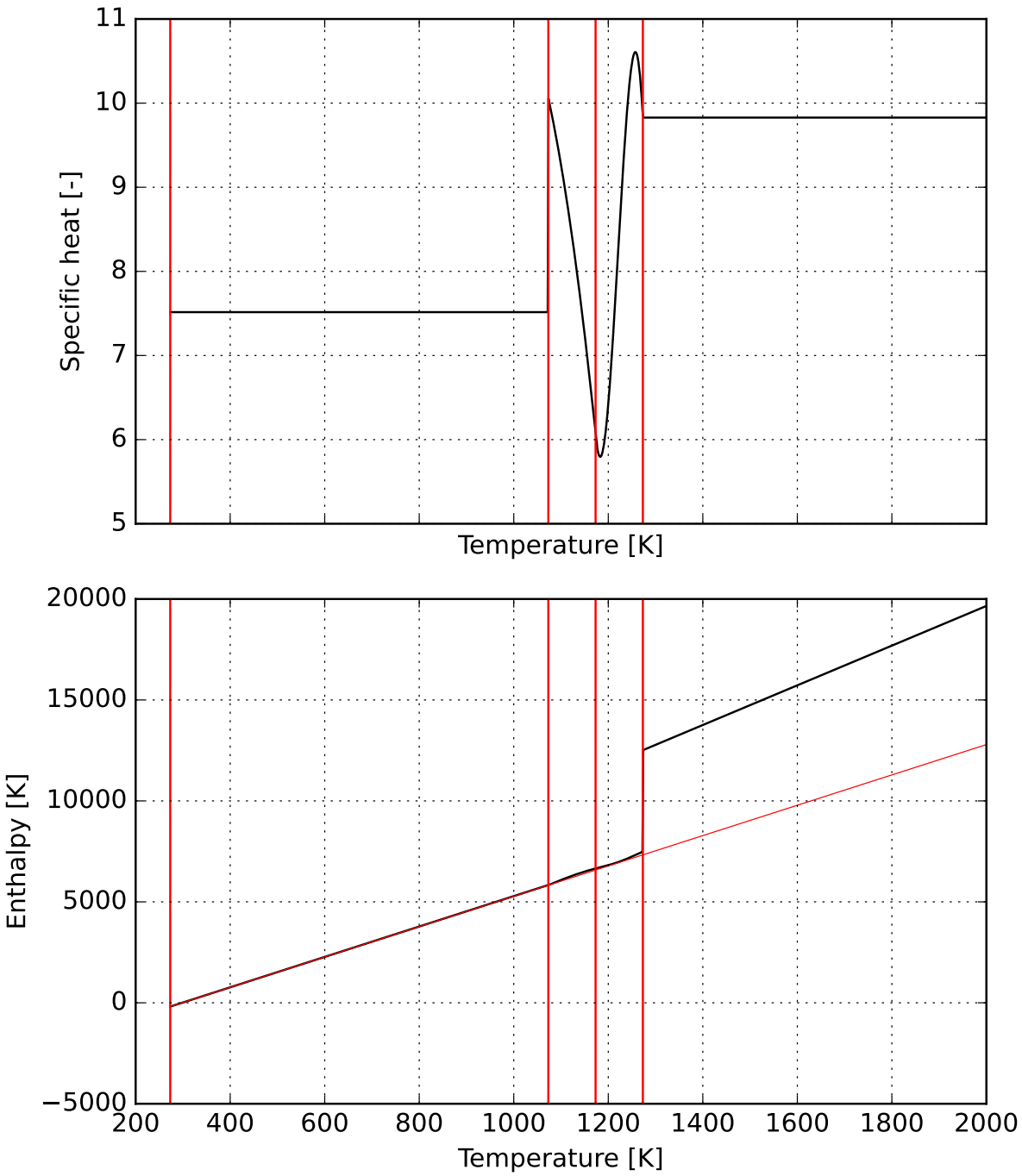


For the second part of the reaction we ensure continuity from the previous one

```
DM([-6.74596e-09, -5.60689e-10, -2.45139e-10, 2.10093e-09, -1.1654e-09])
```


11
10
9
8
7
6
5
Specific heat [-]
Temperature [K]
7500
7400
7300
7200
7100
7000
6900
6800
6700
6600
Enthalpy [K]
1160
1180
1200
1220
1240
1260
1280
Temperature [K]


10.4
10.2
10.0
9.8
9.6
9.4
9.2
Specific heat [-]
Temperature [K]
20000
19000
18000
17000
16000
15000
14000
13000
12000
Enthalpy [K]
1200
1300
1400
1500
1600
1700
1800
1900
2000
Temperature [K]


11
10
9
8
7
6
5
Specific heat [-]
Temperature [K]
20000
15000
10000
5000
0
−5000
Enthalpy [K]
200
400
600
800
1000
1200
1400
1600
1800
2000
Temperature [K]

References

Hanein, Theodore, Frederik P. Glasser, and Marcus N. Bannerman. 2017. "One-Dimensional Steady-State Thermal Model for Rotary Kilns Used in the Manufacture of Cement." *Advances in Applied Ceramics* 116 (4): 207–15. https://doi.org/10.1080/17436753.2017.1303261.

Kee, Robert J., Michael E. Coltrin, Peter Glarborg, and Huayang Zhu. 2017. *Chemically Reacting Flow Theory, Modeling, and Simulation: Theory, Modeling, and Simulation*. Wiley & Sons, Incorporated, John.

Kiradjiev, Kristian B., Svenn Anton Halvorsen, Robert A. Van Gorder, and Sam D. Howison. 2019. "Maxwell-Type Models for the Effective Thermal Conductivity of a Porous Material with Radiative Transfer in the Voids." *International Journal of Thermal Sciences* 145: 106009. https://doi.org/10.1016/j.ijthermalsci.2019.106009.

Sadeghi, Hosein, Simo Hostikka, Guilherme Crivelli Fraga, and Hadi Bordbar. 2021. "Weighted-Sum-of-Gray-Gases Models for Non-Gray Thermal Radiation of Hydrocarbon Fuel Vapors, CH4, CO and Soot." *Fire Safety Journal* 125 (October): 103420. https://doi.org/10.1016/j.firesaf.2021.103420.

Singh, B. P., and M. Kaviany. 1994. "Effect of Solid Conductivity on Radiative Heat Transfer in Packed Beds." *International Journal of Heat and Mass Transfer* 37 (16): 2579–83. https://doi.org/10.1016/0017-9310(94)90295-X.